UserClassification Number: TP39 Student ID: LX20230507206

UDC: 004 Secret Level: None

河海大學

Master's Degree Dissertation

Time Series Load Forecasting using Transformers

Amisi Lumbu Espoir

Supervisor Name: Prof. Lv Xin

Department: College of Computer Science and Software Engineering, Hohai University

Degree Application Level: Master of Engineering

Major: Computer Science and Technology

Submission Date: April 30, 2024 Defense Date : May 27, 2024

Degree-Granting Unit and Date: Hohai University June 20, 2024

Chairman of the Defense Committee: Zhou Xiaofeng

Defense Committee Member: Gao Hongmin Xu Guoyan

Jun 2024 Nanjing · China

Center Seam (Spine)

Time Series Load Forecasting Using Transformers Amisi Lumbu Espoir Hohai University

Classification Number (Chinese Library Classification) TP39 UDC (DDC) 004 Secret Level: None

Author Name Amisi Lumbu Espoir Student ID LX20230507026 Unit Hohai University

Chinese Title of the Dissertation_____

Chinese Subtitle of the Dissertation_____None_____

English Title of the Dissertation Time Series Load Forecasting using Transformers

English Subtitle of the Dissertation_____None_____

Dissertation Language English Dissertation Abstract Language Chinese, English

Number of Dissertation Pages 90 Number of Characters 19,831

Dissertation Keywords Long-term Time Series Forecasting; Load Forecasting; Transformer; PatchTST; Recurrent Neural Network.

Degree Application Level Master of Engineering Major Computer Science and Technology

Research Direction _____Transformer Models Predictions_____

Supervisor Name Professor Lv Xin Supervisor Unit College of Computer Science and Software Engineering, Hohai University

Defense Date:_____, 2026_____

Time Series Load Forecasting using Transformers

Dissertation Submitted to

HoHai University

In Fulfillment of the Requirement

For the Degree of

Master of Engineering

By

Amisi Lumbu Espoir

(College of Computer Science and Software Engineering)

Dissertation Supervisor: Professor Lv Xin

Nanjing, PR China

September 2026

Statement of originality of dissertation:

The dissertation I submitted is my personal research work and research results obtained under the guidance of my supervisor. As far as I know, the dissertation does not contain research results that have been published or written by other people, except for the places where the text is specially marked and acknowledged. My colleagues who work with me have clearly stated in the dissertation and expressed their gratitude for any contributions made to this research. If it is not true, I bear full responsibility.

Dissertation author (signature): year, month day

Authorization Instructions for Dissertation

Hohai University, the Chinese Institute of Science and Technology Information, the National Library of China, and the Chinese Academic Journals (CD-ROM version) electronic magazine have the right to retain the copies or electronic documents of the dissertation submitted by me. Photocopying, reduction, or other reproduction methods may be used to have the dissertation. The content of my electronic document is consistent with the content of the dissertation. Except for confidential papers within the confidentiality period, papers are allowed to be viewed and borrowed. The publication (including publication) of all or part of the dissertation is authorized by the Graduate School of Hohai University.

Dissertation author (signature): year, month day

摘要

长期时间序列预测(LTSF)是电力系统运行的基础，直接关系到发电调度、需求响应与设备维护等关键决策。近年来，以Transformer为代表的自注意力模型在多项公开预测基准上取得了领先性能，但其在有限硬件条件下的实际表现，以及在偏离标准基准数据(如工业现场数据)时的稳定性，尚未得到充分研究。本文以此为切入点，在单块6GB显存的消费级GPU上，针对气象(WTH)、电力变压器温度(ETTh1)、钢铁工业能耗(STEEL)与伦敦智能电表(Smart　Meters)四个数据集，对三种Transformer预测模型(Informer、Autoformer、PatchTST)与四种循环神经网络基线模型(LSTM、BiLSTM、GRU及一种精简单层LSTM配置)进行了系统的对比实验。

研究结果表明，PatchTST在特征维度更高、结构更丰富的数据集(如ETTh1与智能电表数据)上具有明显的准确率优势，而Informer则在气象数据上表现更优，并在所有数据集上均保持最快的训练速度，说明Transformer模型间的优劣并非绝对，而是依赖于具体数据特性。更为重要的是，在钢铁工业数据集上，Informer与Autoformer在训练首个周期即出现数值发散(损失值变为NaN)，且这一现象与模型参数量无关；而全部四种循环神经网络模型及PatchTST均能稳定训练。本文进一步分析发现，其根本原因并非通道混合本身，而在于Informer与Autoformer在跨通道计算注意力或自相关系数时未对数值进行有界处理，而循环网络的门控机制(Sigmoid、Tanh)及PatchTST的通道独立注意力机制则天然具备数值有界性，从而避免了训练失稳。基于以上发现，本文提出了一套兼顾硬件条件与数据特性的模型选择框架，为实际部署中的负荷预测模型选型提供了具有可操作性的依据。

关键词：**长期时间序列预测；负荷预测；Transformer；PatchTST；循环神经网络**

ABSTRACT

Long-term time-series forecasting (LTSF) underpins critical decisions in power system operation, including generation scheduling, demand response, and equipment maintenance. While Transformer-based architectures have recently achieved leading performance on public forecasting benchmarks, their practical behaviour under constrained hardware, and their stability on data that departs from curated benchmarks — such as industrial process data — remain insufficiently studied. This thesis addresses this gap through a systematic, hardware-constrained comparison of three Transformer-based forecasters (Informer, Autoformer, PatchTST) and four recurrent baselines (LSTM, BiLSTM, GRU, and a minimal single-layer LSTM configuration), evaluated on a single 6GB-VRAM consumer GPU across four datasets: Weather (WTH), Electricity Transformer Temperature (ETTh1), Steel Industry Energy Consumption (STEEL), and Smart Meters in London.

The results show that PatchTST achieves a clear accuracy advantage on datasets with richer, higher-dimensional feature structure (ETTh1 and Smart Meters), while Informer performs best on the Weather dataset and is consistently the fastest architecture to train across every dataset — demonstrating that no single Transformer variant is unconditionally superior, and that architectural ranking is data-dependent. More significantly, on the Steel Industry dataset, Informer and Autoformer both diverge numerically (NaN losses) within the first training epoch regardless of model parameter count, while all four recurrent baselines and PatchTST train stably. Further analysis attributes this not to channel-mixing itself, but to the fact that Informer and Autoformer compute unbounded attention or auto-correlation scores across channels, whereas the gating mechanisms (sigmoid, tanh) of the recurrent networks and the channel-independent attention of PatchTST are inherently bounded, avoiding the numerical instability observed. Based on these findings, this thesis proposes a hardware- and data-aware model-selection framework that offers practical, actionable guidance for choosing forecasting architectures in real-world deployment.

Keywords: *long-term time-series forecasting; load forecasting; Transformer; PatchTST; recurrent neural networks.*

Table of Content

摘要..... 6
ABSTRACT.....7
Chapter One Introduction..... 10
1.1 Background and Motivation..... 10
1.1.1 Significance.....11
1.2 Problem Statement.....13
1.3 Key Issues and Research Questions.....15
1.4 Objectives of the Research..... 16
1.5 Scope and limitations.....17
1.6 Thesis organization.....18
Chapter Two Literature Review.....19
2.2 Traditional Methods of Time Series Prediction.....22
2.2.1 Autoregressive Integrated Moving Average(ARIMA).....22
2.2.2 Seasonal ARIMA and Related Seasonal Methods.....23
2.2.3 Vector Autoregression(VAR).....23
2.2.4 Other Classical Methods (VAR/ECM/ETS/Exponential Smoothing)..... 23
2.3 Neural Network-Based Time Series Predictions..... 24
2.3.1 Recurrent Neural Networks (RNNs).....24
2.3.2 Convolutional Neural Networks (CNNs) and Hybrid Architectures.....25
2.4 Transformers-Based Time Series Prediction..... 26
2.4.1 Foundations of the Transformer architecture.....26
2.4.2 Informer..... 27
2.4.3 Autoformer.....28
2.4.4 FEDformer.....28
2.4.5 PatchTST..... 29
2.4.6 Channel Independence and Sub-Series Patching: Recent Advances.....29
2.5 Summary and research gap.....31
Chapter Three Research Methodology.....33
3.1 Overview.....33
3.2 Dataset..... 35
3.3 Preprocessing and Feature Engineering.....39
3.4 Comparative Model Architectures and Core Mechanics.....40
3.4.1 Transformer-Based Models.....41
3.4.2 Recurrent Neural Network Baselines..... 49
3.5 Training Approaches.....54
3.6 Evaluation Metrics.....56
3.7 Experimental Setups..... 57

3.8 Conclusion of Methodological Approach....58
Chapter Four Experimental Results and Discussion....59
4.1 Dataset Description....59
4.2 Preprocessing and Feature Engineering....60
4.3 Model Training....65
4.4 Performance Evaluation....65
4.5 Comparison with RNN-based Models....76
4.6 Discussion of the Findings....78
Chapter Five Conclusion and Future Works....81
5.1 Conclusion....81
5.2 Future works....83
References....84
Acknowledgments....88

Table of Figures

Figure 2.1: Overview of Time-Series Forecasting Methods.....21
Figure 3.1: Research Methodology Flowchart..... 35
Figure 3.2: Standard Transformer Encoder–Decoder Architecture.....42
Figure 3.3: ProbSparse Attention Mechanism.....43
Figure 3.4: Informer Architecture.....44
Figure 3.5: Autoformer Architecture with Series Decomposition.....45
Figure 3.6: Auto-Correlation Mechanism and Period-Based Dependencies.....46
Figure 3.7: Sub-series Patching and Sequence Segmentation..... 47
Figure 3.8: Patching and Temporal Feature Extraction..... 48
Figure 3.9: PatchTST Backbone and Prediction Head..... 49
Figure 3.10: Basic Recurrent Neural Network Unit..... 50
Figure 3.11: LSTM Cell and Gating Structure..... 51
Figure 3.12: Bidirectional LSTM Information Flow.....52
Figure 3.13: GRU Cell and Gating Structure..... 53
Figure 4.1: Sub-series Patching and Sequence Segmentation..... 61
Figure 4.2: PatchTST Architecture and End-to-End Flow..... 63
Figure 4.3: PatchTST Backbone and Prediction Head..... 64
Figure 4.4: WTH: Initial Five-Epoch Calibration Metrics..... 66
Figure 4.5: WTH: Initial Calibration Predictions..... 67
Figure 4.6: WTH: Final Convergence and Metrics at Ten Epochs.....68
Figure 4.7: WTH: Loss and Test Metrics at Ten Epochs.....69
Figure 4.8: WTH: Three-Transformer Comparison.....69
Figure 4.9: ETTh1: Three-Transformer Comparison..... 70
Figure 4.10: ETTh1: Oil Temperature Predictions..... 71
Figure 4.11: STEEL: Initial Three-Transformer Comparison..... 72
Figure 4.12: STEEL: Seven-Model Performance and Training-Time Comparison..... 73
Figure 4.13: STEEL: Seven-Model Energy-Usage Predictions..... 74
Figure 4.14: Smart Meters: Seven-Model Performance and Training-Time Comparison..... 76
Figure 4.15: Smart Meters: Seven-Model Energy-Consumption Predictions..... 78

Chapter One

Introduction

1.1 Background and Motivation

The remarkable success of Transformer architectures in Natural Language Processing (NLP) and Computer Vision (CV) has motivated substantial interest in extending them to time-series data, where the self-attention mechanism's capacity to model long-range dependencies and complex temporal interactions offers a compelling alternative to conventional sequence models [1].

This interest is reinforced by a global energy landscape undergoing rapid transformation. Global electricity demand is projected to grow substantially in the coming decades as digitalization and electrification accelerate [2], intensifying the need for forecasting tools that support grid stability, efficient resource allocation, and the integration of variable renewable generation into modern power systems [3]. Precise long-term load forecasting has consequently become central not only to operational efficiency but to the broader energy transition.

Time-series forecasting has historically relied on statistical methods such as ARIMA together with Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks to predict future values from sequential historical data [4, 5]. LSTM-based models in particular remain widely deployed for load forecasting owing to their comparatively low computational cost and mature tooling [6]. However, Long-Term Time Series Forecasting (LTSF) requires models to map extended historical windows onto extended future horizons, a setting in which recurrent architectures are prone to vanishing or exploding gradients and consequently struggle to retain information across very long sequences.

The Transformer architecture addresses long-range dependency modelling through self-attention rather than recurrence [1]; however, the standard self-attention mechanism incurs quadratic time and memory complexity with respect to input sequence length, and the conventional encoder–decoder structure is prone to information bottlenecks when generating long-horizon forecasts. These limitations motivated a succession of

LTSF-specific Transformer variants: the Informer introduces a ProbSparse self-attention mechanism to reduce this complexity [7]; the Autoformer replaces point-wise attention with a decomposition architecture and auto-correlation mechanism to more effectively isolate trend and seasonal components [8]; and, more recently, PatchTST segments each series into sub-series patches and processes each variable independently, substantially reducing the effective attention cost while improving robustness to distribution shift [9].

Despite these architectural advances, a growing body of subsequent work has questioned whether the reported superiority of Transformer-based LTSF models is unconditional. Zeng et al. demonstrated that a considerably simpler linear model could match or exceed several Transformer variants on standard benchmarks [10], and related work has shown that processing each variable independently — a central design choice in PatchTST — reflects a genuine trade-off between representational capacity and robustness rather than a strict improvement [11]. Moreover, the datasets on which these comparisons are typically conducted — meteorological records, transformer temperature series, and similar benchmarks — exhibit continuity and moderate scale ranges that may not be representative of real industrial energy consumption data, and such comparisons are rarely conducted under the memory and compute constraints faced by non-server-class deployment. This gap between idealized benchmark evaluation and practical deployment realities constitutes the central motivation of this research.

1.1.1 Significance

Accurate long-term load forecasting has significant academic, economic, environmental, and industrial importance. It is an essential component of smart-grid operations because it supports efficient resource allocation, effective demand-response management, and the stable integration of renewable energy generation . By accurately predicting future electricity demand, grid operators can improve system planning, reduce operational uncertainty, and maintain a reliable balance between energy supply and consumption.

For industrial organizations and utility providers, forecasting models must not only achieve high predictive accuracy but also operate within the computational resources available in real-world deployment environments. In practice, these resources are often

considerably more limited than the server-class hardware used to produce state-of-the-art results on benchmark datasets. Therefore, the practical significance of this research lies in evaluating forecasting architectures under a consistent and constrained hardware budget rather than assuming access to unlimited computational capacity.

Overall, accurate and computationally efficient load forecasting offers several important benefits:

- It enables energy providers and industrial operators to optimize generation planning, production scheduling, and resource allocation.
- It supports demand-response strategies by helping organizations anticipate periods of high or low electricity consumption.
- It facilitates the stable integration of renewable energy sources by improving the coordination between variable generation and future energy demand.
- It reduces energy waste and operational costs by enabling more informed scheduling and minimizing the need for inefficient corrective actions.
- It benefits consumers by contributing to more reliable energy services and potentially reducing costs associated with inefficient energy utilization.
- It promotes environmental sustainability by supporting the efficient use of energy and reducing unnecessary consumption.

The research also has theoretical significance. It contributes to the ongoing discussion regarding the role of channel independence and sub-series patching in long-term time-series forecasting. In addition, it examines whether the greater representational capacity of self-attention mechanisms produces sufficient accuracy gains to justify their increased training and deployment costs . This is particularly important because model complexity should be evaluated not only in terms of predictive performance but also in relation to computational efficiency and practical usability.[9][11]

Another important contribution concerns the limited research on forecasting model behavior when applied to industrial energy-consumption data. Unlike the relatively smooth and regular benchmark datasets commonly used in existing studies, industrial energy data may exhibit extreme differences in scale, irregular statistical patterns, and discontinuous operating regimes[12]. Investigating model performance in this context can provide a more realistic understanding of the strengths and limitations of Transformer-based forecasting

architectures. The study also documents a failure mode that is directly relevant to the safe deployment of Transformer-based forecasters in industrial environments [13].

Taken together, this research has both theoretical and practical significance. Theoretically, it provides a rigorous comparison of forecasting architectures with different levels of complexity by jointly considering predictive accuracy, computational cost, robustness, and failure behavior. Practically, it offers evidence-based guidance for selecting an appropriate forecasting model according to the characteristics of the available data and the computational constraints of the deployment environment, rather than relying solely on results obtained under idealized benchmark conditions.

1.2 Problem Statement

Although Transformer-based models hold considerable promise for long-term time-series forecasting, several critical challenges limit the direct transferability of their reported benchmark performance to practical load-forecasting deployments. First, the standard self-attention mechanism is not computationally efficient and imposes substantial memory overhead due to its quadratic complexity with respect to input sequence length [1], a constraint that becomes acute under the limited hardware budgets available to many utilities and research groups. Second, although efficient variants such as Informer, Autoformer, and PatchTST have reported strong results on their original evaluation benchmarks [7, 8, 9], their robustness and generalization to energy data exhibiting the extreme scale heterogeneity and discontinuous behaviour characteristic of industrial consumption remain largely unexamined. Third, existing comparative studies rarely report forecasting accuracy jointly with the computational cost — training time, memory footprint, and parameter count — required to achieve it, making the trade-off between the two difficult to assess for practitioners operating under hardware constraints. Finally, where Transformer-based models fail outright rather than merely underperforming, the literature offers limited diagnostic detail regarding the data conditions responsible, restricting the ability of practitioners to anticipate or mitigate such failures in deployment. This study addresses these gaps by conducting a systematic, jointly reported evaluation of Transformer-based and recurrent forecasting architectures across datasets of varying

statistical regularity, under a constrained and explicitly documented hardware budget, and extends the assessment to a failure case on industrial data that is largely absent from the existing literature.

1.3 Key Issues and Research Questions

Long-term time-series forecasting has become an essential task in energy management and industrial operations, as it supports applications such as demand planning, resource allocation, and predictive decision-making. Although Transformer-based models have achieved promising results in sequence forecasting, several intrinsic challenges remain, particularly when they are deployed under practical computational constraints. These challenges include the high computational cost of processing long sequences, limited robustness to heterogeneous industrial data, and the lack of a comprehensive understanding of the conditions under which different forecasting architectures succeed or fail. To address these challenges, the proposed study aims to explore and answer the following research questions:

RQ1. Under a fixed, constrained hardware budget, how do Transformer-based models (Informer, Autoformer, PatchTST) compare to recurrent baselines (LSTM, BiLSTM, GRU) in forecasting accuracy and computational efficiency for long-term load forecasting?

RQ2. To what extent do Transformer-based models demonstrate robustness and generalization when applied to industrial energy consumption data exhibiting extreme scale heterogeneity and discontinuous behaviour, as opposed to standard meteorological or residential benchmarks?

RQ3. Where Transformer-based models fail to train or produce degenerate predictions, what data characteristics are associated with these failures, and what mechanisms plausibly account for them?

RQ4. Can the findings from these comparisons be synthesized into practical, hardware- and data-aware guidance for selecting a forecasting architecture under a given combination of data characteristics and computational constraints?

To address these research questions, the study evaluates several efficient Transformer architectures designed to mitigate the computational challenges associated with

long-sequence forecasting. In particular, Informer's ProbSparse attention mechanism and PatchTST's sub-series patching strategy are examined for their ability to reduce the time and memory requirements associated with processing long input sequences. The study also includes Autoformer, Informer, and PatchTST, alongside recurrent baselines such as LSTM, BiLSTM, GRU, and a minimal single-layer LSTM configuration. Their forecasting accuracy, training behavior, inference efficiency, and computational requirements are evaluated under a single, fixed hardware budget to ensure a consistent and practically meaningful comparison.[5][6][7][9]

Furthermore, the study employs appropriate datasets, forecasting metrics, computational measurements, and experimental procedures that align with its research objectives. By jointly examining predictive accuracy, computational efficiency, robustness, generalization, and failure conditions, the research seeks not only to compare competing forecasting architectures but also to clarify the practical trade-offs involved in deploying them in real-world industrial environments. It is expected that the findings will provide useful insights into the relationship between data characteristics, model architecture, and hardware limitations.

In conclusion, the contributions of this study include addressing the computational limitations of long-sequence forecasting, evaluating the generalization of efficient Transformer models on industrial energy-consumption data, and providing a rigorous comparison with established recurrent baselines. The study also seeks to identify the data properties associated with model failure and to formulate practical recommendations for selecting forecasting architectures according to available computational resources and dataset characteristics. Limitations and future directions are also discussed, highlighting opportunities for further research into efficient, robust, and deployment-oriented time-series forecasting.

1.4 Objectives of the Research

The main objective of this study is to investigate the performance of Transformer-based models relative to recurrent baselines for long-term energy load forecasting under realistic hardware constraints. Specifically, the research aims to:

1. Employ three Transformer-based models (Informer, Autoformer, PatchTST) and four

recurrent baselines (LSTM, BiLSTM, GRU, and a minimal single-layer LSTM configuration) for time series predictions.

2. Design a unified experimental protocol, executed on a single, explicitly documented hardware configuration, to jointly evaluate predictive accuracy and computational efficiency across all seven models.

3. Train and evaluate the models on preprocessed data using consistent optimization procedures, and analyze the resulting accuracy and efficiency metrics in detailAnalyze the benefits and limitations of using Transformers based models for Time series prediction.

1.5 Scope and limitations

This research concentrates on long-term energy load forecasting, comparing three Transformer-based models (Informer, Autoformer, PatchTST) against four recurrent baselines (LSTM, BiLSTM, GRU, and a minimal single-layer LSTM configuration) across four datasets spanning a range of statistical regularity — from continuous meteorological and residential consumption data to discontinuous industrial energy consumption data. All experiments are conducted on a single NVIDIA RTX 2060 GPU with 6GB of VRAM, a configuration selected to represent constrained, non-server-class hardware rather than to establish the maximum achievable performance of any given architecture.

While the principles and findings of this research may be transferable to other time-series forecasting domains, such as traffic flow or financial markets, this study exclusively addresses energy consumption data as its use case, and the selection of specific models and baselines defines the boundary of the comparison; generalization to other Transformer variants or baseline models is beyond its immediate scope. Hyperparameters were held fixed across comparable model categories to preserve comparability rather than individually tuned to optimality, which may understate the best-case performance achievable by any single architecture. Where a failure mode is identified and an explanatory mechanism proposed, this thesis treats that mechanism as a hypothesis supported by the available evidence rather than a definitively established causal finding, pending confirmatory ablation experiments identified as future work. Finally, reliance on publicly available datasets means the study is constrained by their characteristics, quality, and

documentation.

1.6 Thesis organization

The thesis is organized as follows:

Chapter 1: Introduction - Provides the background of the study, its significance, the problem statement, key issues and research questions, objectives, and the scope and limitations of the research..

Chapter 2: Literature Review - Offers a comprehensive overview of existing literature on time-series forecasting methods, the evolution of Transformer models for time-series analysis, and the ongoing debate concerning channel independence, sub-series patching, and the accuracy–efficiency trade-off in long-term forecasting..

Chapter 3: Methodology - Details the datasets, the architectures of the Transformer-based models (Informer, Autoformer, PatchTST) and the recurrent baselines (LSTM, BiLSTM, GRU, minimal LSTM), the unified training protocol, and the evaluation metrics used.

Chapter 4: Results and Discussion - Presents the experimental setup, dataset characteristics, the joint accuracy–efficiency evaluation of all seven models, and a detailed failure analysis on the industrial dataset..

Chapter 5: Conclusion and Future Work - Summarizes the study's key findings and contributions, synthesizes the hardware- and data-aware model-selection framework, acknowledges limitations, and outlines directions for future research, including the confirmatory experiments required to validate the proposed failure mechanism.

Chapter Two

Literature Review

2.1 Overview

Time-series forecasting has long served as a cornerstone of decision-making in domains ranging from energy systems and financial markets to climate science and industrial operations. With the accelerating digitization of infrastructure and the proliferation of high-resolution sensor data, the demand for accurate, scalable, and robust forecasting models has intensified, particularly as global electricity demand continues to grow and power systems incorporate increasing shares of variable renewable generation [2]. Traditional statistical approaches, while interpretable and theoretically well-grounded, often falter when confronted with the nonlinearity, high dimensionality, and long-range temporal dependencies characteristic of modern real-world datasets, including the electrical load data central to this thesis [3].

The advent of deep learning has ushered in a new era of forecasting methodology, in which neural architectures — recurrent networks, convolutional networks, and, more recently, Transformer-based models — have demonstrated substantial advances in predictive performance. Recurrent neural networks and their gated variants (LSTM, GRU) addressed the linearity constraints of classical statistical models by learning nonlinear temporal dependencies directly from data [5, 6], while convolutional and hybrid architectures introduced parallelizable alternatives capable of efficiently extracting local temporal patterns. Most recently, Transformer architectures, originally developed for natural language processing, have rapidly permeated the time-series community owing to their capacity to model long-range interactions in parallel, without the sequential bottlenecks inherent to recurrent computation [1]. This shift has catalyzed the emergence of specialized Transformer variants tailored explicitly for Long-Term Time-Series Forecasting (LTSF), including Informer, Autoformer, FEDformer, and PatchTST, each introducing architectural refinements intended to address the vanilla Transformer's quadratic

computational complexity, memory inefficiency, and misalignment with time-series-specific structures such as trend, seasonality, and sparsity [7, 8, 9].

The literature review chapter discusses various types of time series predictions methods, as illustrated in the following diagram:

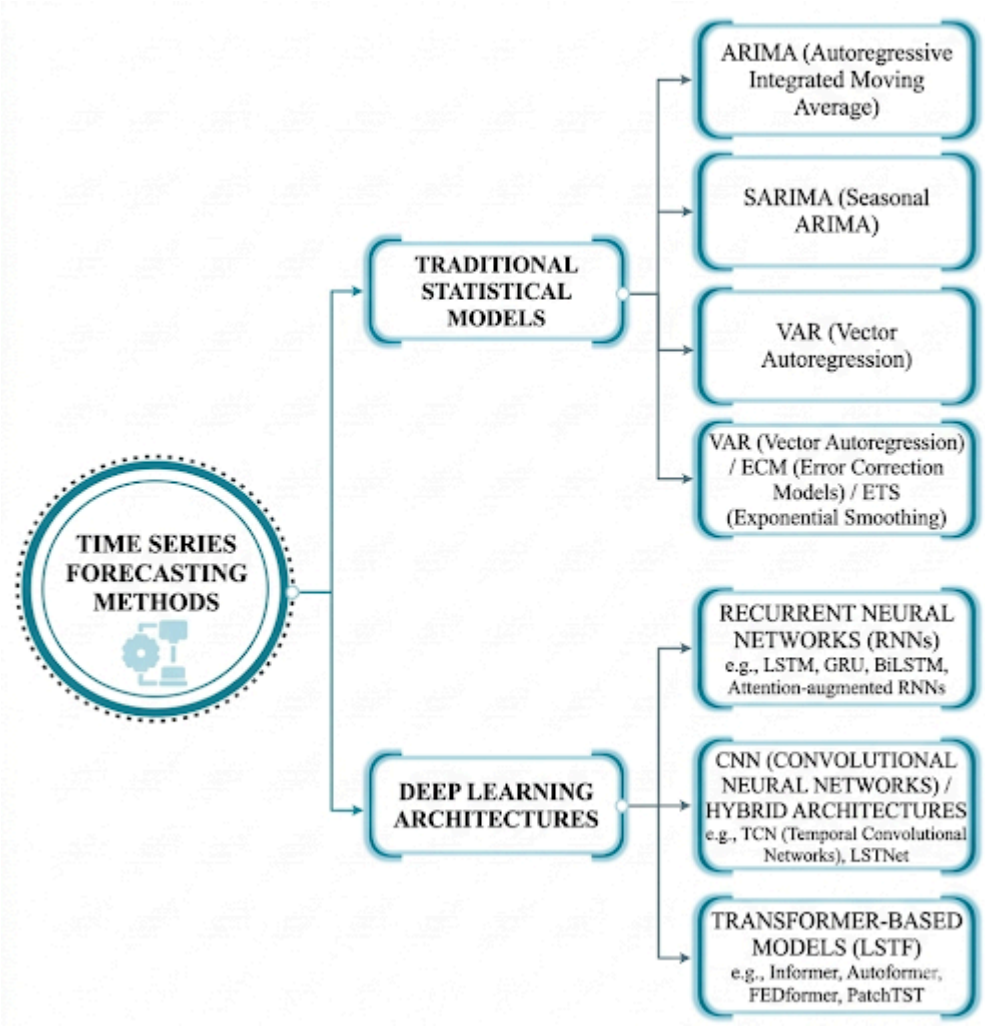


Figure 2.1: Overview of Time-Series Forecasting Methods

This chapter provides a systematic review of the evolution of time-series forecasting methods, beginning with classical statistical models, progressing through neural network-based approaches, and culminating in the current frontier of Transformer-based architectures, including the post-2023 literature on channel independence and sub-series patching that directly informs the design of this study. By situating these developments within the broader literature and highlighting empirical findings from recent work, the chapter identifies both the theoretical motivations and practical trade-offs that define the modern forecasting landscape, and concludes by articulating the specific research gaps that motivate the present study.

2.2 Traditional Methods of Time Series Prediction

Traditional time-series forecasting methods are rooted in statistical modelling principles that rely on assumptions of stationarity, linearity, and well-defined stochastic processes [4]. These approaches have historically underpinned applications in energy systems, finance, and climate forecasting, offering interpretable models that provide clear insight into temporal dynamics, and they remain useful for establishing baseline performance on stable datasets. However, their linear assumptions, limited capacity to represent multivariate dependencies, and difficulty with long-horizon prediction increasingly restrict their applicability in modern, high-frequency, high-dimensional settings such as smart grids and industrial energy monitoring — limitations that have driven the field toward the nonlinear, data-driven approaches reviewed in Sections 2.3 and 2.4.

2.2.1 Autoregressive Integrated Moving Average(ARIMA)

The Autoregressive Integrated Moving Average (ARIMA) model, formalized by Box and Jenkins, remains one of the most widely used classical forecasting methods [14]. It integrates three components: autoregression (AR), which models the relationship between an observation and its own past values; differencing (I), applied to achieve stationarity in non-stationary series; and moving averages (MA), which account for correlation among past forecast errors. ARIMA is particularly effective for univariate series exhibiting stable statistical properties, and its interpretable structure makes it a preferred choice where model transparency is valued. However, its reliance on linear assumptions limits performance in the presence of nonlinear dynamics or high-frequency fluctuations; Hippert et al. demonstrated that while ARIMA could reliably forecast short-term electricity load, its accuracy deteriorated markedly under volatile consumption patterns or high-dimensional input features, motivating the need for more flexible modelling approaches of the kind reviewed later in this chapter [15].

.

2.2.2 Seasonal ARIMA and Related Seasonal Methods

Building upon ARIMA, Seasonal ARIMA (SARIMA) incorporates explicit seasonal components to account for periodic variation, introducing additional parameters to model recurring patterns over defined seasonal intervals, making it well suited to datasets with strong cyclical behaviour such as daily or weekly electricity consumption. Taylor's work on triple seasonal methods for short-term electricity demand forecasting demonstrated that explicitly modelling multiple superimposed seasonal cycles — daily, weekly, and annual — could yield meaningful accuracy improvements over standard seasonal decomposition on utility load data, while also showing that performance degraded during periods of abrupt demand shifts not captured by the fitted seasonal structure [16]. This sensitivity to structural change is a recurring theme that motivates the nonlinear approaches discussed subsequently.

2.2.3 Vector Autoregression(VAR)

The Vector Autoregression (VAR) model extends the autoregressive framework to multivariate settings, modelling each variable as a linear combination of its own past values and those of other variables in the system. This property makes VAR useful where multiple interdependent series influence one another, such as electricity load and ambient temperature. Lütkepohl's comprehensive treatment of multiple time-series analysis illustrates VAR's utility in economic and energy contexts while emphasizing its constraints: the “curse of dimensionality” reduces scalability as the number of variables grows, and the model's linear interaction assumption limits its effectiveness on the nonlinear relationships present in many modern datasets, including the multivariate industrial energy data considered in this thesis [17].

2.2.4 Other Classical Methods (VAR/ECM/ETS/Exponential Smoothing)

Beyond ARIMA, SARIMA, and VAR, several additional statistical approaches have been widely used in traditional forecasting. Error Correction Models capture long-term equilibrium relationships among cointegrated variables, following the foundational

cointegration framework of Engle and Granger, and are valuable in multivariate systems where series move together over time [19]. Exponential Smoothing State Space Models (ETS), formalized within a unified state-space framework by Hyndman et al., rely on weighted averages of past observations and can incorporate trend and seasonal components, allowing them to adapt to gradual change while remaining computationally efficient for short-term forecasting in stable environments [18]. Collectively, these methods provide a suite of tools for modelling linear or quasi-linear temporal dynamics, offering complementary capabilities depending on dataset characteristics, even as contemporary forecasting has moved toward the data-driven, nonlinear approaches reviewed next.

2.3 Neural Network-Based Time Series Predictions

The limitations of traditional statistical methods — particularly their linearity assumptions and difficulty representing long-range dependencies — motivated the development of neural network-based forecasting approaches capable of modelling complex nonlinear relationships and multivariate dependencies across diverse domains. The evolution of neural forecasting methods can be broadly categorized into recurrent architectures and convolutional or hybrid architectures, each addressing specific challenges in temporal modelling.

2.3.1 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks were among the first neural architectures designed specifically for sequential data. By maintaining internal memory states that encode information from previous time steps, RNNs capture temporal dependencies crucial to accurate forecasting; however, early RNN variants suffered from vanishing and exploding gradient problems that limited their ability to learn long-range dependencies. Long Short-Term Memory (LSTM) networks were introduced specifically to overcome this limitation through a gated memory-cell design that regulates the flow of information through the network, allowing relevant information to be retained over substantially longer sequences than a standard RNN [5]. Gated Recurrent Units (GRUs) later offered a simplified gating variant with fewer parameters, achieving comparable performance at reduced computational cost [22]. LSTM-based models in particular have been extensively

applied to load forecasting; Kong et al. demonstrated that an LSTM recurrent architecture could reliably capture the nonlinear temporal dynamics of residential electricity consumption, establishing LSTM as a strong and computationally efficient baseline for load-forecasting research that this thesis adopts accordingly [6].

Extensions such as Bidirectional LSTMs (BiLSTMs) allow a model to access both past and future context during training by processing the sequence in both temporal directions, improving predictive accuracy for sequences with dependencies that are not purely causal [21]. Attention-augmented recurrent architectures further enhance model capability by allowing the network to weight the most relevant segments of historical input dynamically; Sehovac and Grolinger showed that sequence-to-sequence LSTM architectures equipped with attention significantly outperformed standard recurrent baselines in electricity load forecasting, underscoring the value of dynamic temporal weighting even within a fundamentally recurrent architecture [20]. Despite these refinements, RNN-family models face an inherent limitation: their sequential processing impedes parallelization, training is comparatively slow, and very long-range dependencies remain difficult to capture in high-frequency or irregular datasets — the central motivation for the attention-based architectures reviewed in Section 2.4..

2.3.2 Convolutional Neural Networks (CNNs) and Hybrid Architectures

Convolutional neural networks (CNNs), originally developed for spatial data such as images, have been successfully adapted to time-series forecasting through one-dimensional convolutions applied along the temporal dimension. Bai et al. showed that Temporal Convolutional Networks (TCNs), which use dilated causal convolutions to expand the receptive field, could match or exceed recurrent architectures across a range of sequence-modelling tasks while offering more stable gradients and fully parallelizable computation, making CNN-based models considerably more efficient than sequential RNNs for long input sequences [23]. Hybrid architectures such as LSTNet, introduced by Lai et al., combine convolutional layers for extracting local, short-term patterns with recurrent layers for modelling longer-term dependencies, and were shown to be particularly effective on multivariate forecasting tasks by leveraging convolution for feature extraction and recurrence for sequential memory [24]. Empirical evidence from this line of work suggests

that convolutional architectures excel at capturing repeating seasonal and cyclical structure and are less prone to gradient instability than plain RNNs, though their effectiveness can decline on irregularly sampled data or extremely long sequences without hybridization — a limitation that motivated the shift toward fully attention-based modelling described next.

2.4 Transformers-Based Time Series Prediction

While RNN and CNN-based architectures advanced the capability of neural forecasting, both face challenges in capturing very long-range dependencies or processing extremely long sequences efficiently. Transformer-based models, originally developed for natural language processing, replace sequential recurrence with self-attention mechanisms, enabling parallel processing and direct modelling of interactions across all time steps [1]. However, adapting Transformers to time-series forecasting revealed challenges specific to the domain, including quadratic computational complexity, inefficient long-horizon prediction, and a lack of alignment with time-series-specific structure such as seasonality, trend, and sparsity. The following subsections review the foundational architecture and the principal LTSF-specific variants developed to address these challenges.

2.4.1 Foundations of the Transformer architecture

A foundational understanding of the Transformer architecture is essential to situating the LTSF-specific variants reviewed subsequently. The vanilla Transformer, introduced by Vaswani et al., follows an encoder–decoder structure in which both encoder and decoder are composed of multiple identical blocks; each encoder block consists of a multi-head self-attention module and a position-wise feed-forward network, while each decoder block additionally inserts a cross-attention module between the self-attention and feed-forward components [1].

Unlike LSTM or RNN architectures, the vanilla Transformer contains no recurrence and instead relies on positional encoding, added to the input embeddings, to represent sequence order. Absolute positional encodings assign each position a fixed or learned embedding vector, with learned variants offering greater flexibility, as demonstrated in the BERT architecture of Devlin et al. [25]. Relative positional encoding schemes instead

model the pairwise relationship between input positions directly; Shaw et al. proposed adding a learnable relative positional embedding to the attention keys, motivated by the intuition that relative position is often more informative than absolute position for sequence modelling [27]. Convolutional alternatives to positional encoding, such as the convolutional sequence-to-sequence architecture of Gehring et al., demonstrated that positional information could also be embedded implicitly through convolutional structure rather than explicit encoding [26].

The core computational primitive of the Transformer is scaled dot-product attention over a Query-Key-Value (QKV) representation, in which queries and keys are compared to produce attention weights that are applied to the values; multi-head attention extends this by computing several such attention functions in parallel over learned projections of the input, allowing the model to attend to different representation subspaces simultaneously [1]. Each attention sub-layer is followed by a position-wise feed-forward network, and residual connections combined with layer normalization are applied around each sub-layer to stabilize training in deep stacks of these blocks. It is this attention mechanism, and specifically its quadratic cost with respect to sequence length, that motivated the efficiency-oriented variants reviewed below.

2.4.2 Informer

Informer introduces ProbSparse self-attention, which computes only the most informative query-key pairs rather than the full pairwise attention matrix, reducing attention complexity from $O(L^2)$ to $O(L \log L)$ with respect to sequence length L [7]. The model further incorporates self-attention distilling, which progressively down-samples the sequence representation within the encoder to reduce memory usage across stacked layers, and a generative-style decoder capable of predicting the entire output sequence in a single forward pass rather than through iterative, step-by-step decoding. Zhou et al. demonstrated Informer's superiority over the standard Transformer on electricity load and traffic forecasting benchmarks, particularly for sequences exceeding several hundred time steps, establishing it as one of the first LTSF-specific Transformer variants to gain wide adoption in the load-forecasting literature [7].

2.4.3 Autoformer

Autoformer addresses long-horizon forecasting by explicitly decomposing the input sequence into trend-cyclical and seasonal components via moving-average-based decomposition blocks embedded directly within the model architecture. Its auto-correlation mechanism replaces standard point-wise attention with a periodicity-based weighting scheme that identifies and aggregates sub-series at the most relevant periodic lags, allowing the model to capture repeating temporal patterns more efficiently than dense self-attention while simultaneously reducing computational cost [8]. Wu et al. reported that Autoformer achieved strong performance on energy and traffic datasets exhibiting pronounced cyclical behaviour, outperforming both the standard Transformer and Informer on several long-horizon benchmarks, though — as the experimental results in Chapter 4 of this thesis show — its slower initial convergence relative to Informer on some datasets suggests that benchmark-level superiority does not always translate into faster or more stable training under constrained compute budgets [8].

2.4.4 FEDformer

FEDformer operates in the frequency domain, applying the Fast Fourier Transform to identify a compact set of dominant spectral components rather than processing the sequence entirely in the time domain. Zhou et al. showed that this frequency-domain formulation, combined with a mixture-of-experts decomposition mechanism, reduces sequence-modelling complexity toward linear time while improving robustness to noise, allowing FEDformer to effectively capture periodicity in energy, traffic, and industrial datasets, particularly where high-frequency fluctuations are present [28]. FEDformer is included in this literature review for architectural completeness, though it was not among the models empirically evaluated in this thesis; its frequency-domain approach nonetheless represents an important complementary direction to the patch-based and decomposition-based methods that are evaluated.

2.4.5 PatchTST

PatchTST introduces a patching strategy that segments each input series into shorter sub-sequences, or patches, each of which is treated as a single token within the attention mechanism, reducing the effective sequence length from L to N ≪ L patches and correspondingly reducing attention complexity toward $O(N^2)$ [9]. Combined with channel-independent processing — in which each variate of a multivariate series is treated as an independent univariate input stream rather than being fused with other variates at the input stage — PatchTST achieves improved accuracy, faster training, and lower memory consumption relative to earlier Transformer variants. Nie et al. demonstrated that PatchTST consistently outperformed prior Transformer models and recurrent baselines on high-dimensional, high-variance benchmark datasets including Weather and Electricity Transformer Temperature (ETTh1), particularly under limited GPU memory budgets, a finding directly consistent with the accuracy and efficiency results obtained for PatchTST in this thesis [9].

2.4.6 Channel Independence and Sub-Series Patching: Recent Advances

The channel-independence design choice introduced by PatchTST has, since 2023, become an active research axis in its own right, with subsequent work examining both its benefits and its costs. Zeng et al. challenged the premise that architectural complexity translates into forecasting accuracy, showing that a considerably simpler linear model could match or exceed several Transformer variants, including early patch-based models, on standard LTSF benchmarks — a result that directly motivates this thesis's emphasis on jointly reporting accuracy and computational cost rather than accuracy alone [10]. In direct response to the channel-independence question, Han et al. formalized the underlying trade-off explicitly, demonstrating that channel-independent training sacrifices cross-variate representational capacity in exchange for improved robustness to distribution shift, a finding with direct relevance to the heterogeneous, multivariate industrial data examined in Chapter 4 of this thesis [11].

Other recent work has pushed in the opposite direction, arguing that discarding cross-variate information entirely is suboptimal for datasets with strong inter-variable

structure. Zhang and Yan's Crossformer explicitly models cross-dimension dependency alongside temporal dependency through a two-stage attention mechanism [29], while Liu et al.'s iTransformer inverts the conventional Transformer input representation, treating each variate as a token and each time step as a feature, to allow the model to attend across variates directly [30]. Wang et al.'s CARD architecture proposes channel-aligned attention specifically to recover useful inter-variable structure that channel-independent processing discards [31], and Ilbert et al. attribute part of the accuracy gap between simple linear models and Transformers to the sharper loss landscapes induced by channel-independent training, proposing sharpness-aware minimization as a mitigation [32]. Chen et al.'s Pathformer further generalizes the patching concept introduced by PatchTST into a multi-scale, adaptive patching scheme capable of selecting patch granularity dynamically based on the input series [33]. Reversible Instance Normalization (RevIN), proposed by Kim et al., addresses a related but distinct problem — per-window distribution shift — by normalizing each input window prior to processing and reversing the transformation on the output, and is now used within PatchTST and several subsequent architectures specifically to improve robustness to non-stationary input statistics [13]. Taken together, this body of work indicates that channel independence, sub-series patching, and instance normalization are not settled design choices but active trade-offs whose appropriateness depends on the statistical structure of the target data — precisely the dependency this thesis investigates empirically using both regular benchmark data and industrial energy data with markedly different statistical properties. Recent surveys by Wen et al. and Su et al. provide comprehensive taxonomies of this rapidly expanding design space but explicitly note the continued scarcity of controlled efficiency comparisons and documented failure cases in the published literature, a gap this thesis directly addresses [34, 35].

2.5 Summary and research gap

The evolution of time-series forecasting has progressed from linear statistical models (Sections 2.2.1–2.2.4) to nonlinear neural architectures — recurrent and convolutional (Section 2.3) — and, most recently, to purpose-built Transformer architectures (Section 2.4). Each paradigm shift has addressed specific limitations of its predecessor: neural architectures overcame the linearity constraints of ARIMA-family models; convolutional and hybrid architectures improved short-term pattern extraction and parallel computational efficiency relative to recurrence; Transformers overcame the sequential processing bottleneck of RNNs through self-attention; and specialized variants such as Informer, Autoformer, and PatchTST refined the Transformer specifically for the demands of long-term forecasting. Most recently, the channel-independence and patching literature reviewed in Section 2.4.6 has begun to interrogate whether these architectural refinements represent unconditional improvements or context-dependent trade-offs.

Despite this progress, four research gaps remain evident from the literature reviewed in this chapter:

- Hardware-Aware Model Selection: Real-world deployment, particularly on constrained hardware with limited GPU memory, requires explicit joint consideration of architectural accuracy and computational efficiency, yet the published comparative literature rarely reports both under a controlled, shared protocol [9, 10].
- Data-Driven Model Suitability: Forecasting performance depends critically on data characteristics such as continuity, sparsity, and scale heterogeneity, and the behaviour of Transformer-family architectures on industrial data exhibiting these properties remains largely unexamined in the reviewed literature [11, 12].
- Accuracy–Efficiency Trade-offs: Transformers frequently report superior accuracy on curated benchmarks, while recurrent architectures retain clear advantages in training speed, memory footprint, and parameter efficiency; explicit, quantified guidance on navigating this trade-off remains limited [9, 10].
- Failure-Mode Documentation: Instances in which state-of-the-art architectures fail

outright, as opposed to merely underperforming, are rarely documented with sufficient diagnostic detail to support practical risk assessment, despite the surveys reviewed in Section 2.4.6 explicitly identifying this as an open gap [34, 35].

This thesis addresses these gaps directly by conducting a controlled, multi-dataset evaluation of three Transformer architectures (Informer, Autoformer, PatchTST) alongside four recurrent baselines (LSTM, BiLSTM, GRU, and a minimal single-layer LSTM configuration), executed under a single, explicitly documented hardware budget representative of constrained, non-server-class deployment. Through joint quantitative benchmarking, qualitative prediction-fidelity analysis, and a detailed failure analysis on an industrial energy dataset exhibiting the extreme scale heterogeneity and discontinuous behaviour identified as underexamined above, the study provides actionable, evidence-based guidance for both researchers and practitioners navigating the expanding landscape of time-series forecasting architectures.

Chapter Three

Research Methodology

This chapter is devoted to the research methodology of the study. It provides an overview of the research methodology, the employed dataset, preprocessing & feature engineering tasks, model architecture, evaluation metrics, and experimental setups.

3.1 Overview

This chapter presents, in full detail, the research methodology used to investigate the comparative performance of Transformer-based and recurrent forecasting architectures for long-term time series forecasting (LTSF), with a particular focus on electricity load forecasting under realistic hardware constraints. It documents the datasets selected and their statistical properties, the preprocessing and feature-engineering pipeline applied uniformly to them, the internal mechanics of each of the seven models evaluated — including their mathematical formulation and, for every architecture, an accompanying structural diagram — the training protocol followed, and the evaluation framework used to compare predictive accuracy against computational cost. The chapter follows a structured, reproducible approach intended to ensure that the results reported in Chapter 4 are robust, internally consistent, and relevant to real-world deployment conditions rather than artefacts of an uncontrolled experimental design.

The central objective of this study is to assess the practical viability of three Transformer-based architectures — Informer, Autoformer, and PatchTST [7, 8, 9] — against four recurrent baselines — LSTM, BiLSTM, GRU, and a deliberately minimal single-layer LSTM configuration [5, 21, 22] — for electricity load forecasting. The research places particular emphasis on the trade-off between predictive accuracy, model complexity, and computational efficiency, and on how this trade-off is affected by the constrained hardware budgets typical of small research groups and edge deployments, rather than the server-class hardware assumed in much of the published LTSF literature [9, 10]. All experiments reported in this thesis were executed on a single ASUS ROG Zephyrus G14 (GA401IV) laptop equipped with an NVIDIA GeForce RTX 2060 (Max-Q, 6GB

VRAM), a configuration chosen specifically to represent this class of constrained, non-server-class hardware.

The experimental design follows a systematic, staged approach. Four publicly available datasets, spanning meteorological, industrial, and residential domains, were selected to test model generalizability across a range of statistical regularity — from the smooth, continuous behaviour of weather data to the extreme scale heterogeneity and discontinuity of industrial energy data [12]. After a unified preprocessing and feature-engineering pipeline was applied, all seven models were trained under matched conditions on each dataset, with predictive accuracy (MAE, MSE, RMSE) reported jointly with operational cost (training time, parameter count, and VRAM occupancy), consistent with the hardware-aware evaluation framework motivated in Chapters 1 and 2 [10, 11].

A methodological principle that runs throughout this chapter, and that distinguishes this study from much of the comparative LTSF literature, is that architectural description and empirical configuration are kept explicitly separate. Where a structural diagram in this chapter illustrates a model's general mechanism using representative input dimensions drawn from a reference implementation, this is stated plainly, and the reader is directed to the actual experimental configuration used in this thesis — detailed fully in Section 3.5 and Section 3.7 — for the precise Lookback, Horizon, and batch-size values used to generate the results reported in Chapter 4. This separation is necessary because illustrative shapes convenient for explaining a mechanism (for instance, a short lookback window that fits legibly in a diagram) are not always identical to the values used in full-scale experimentation on 6GB VRAM hardware.

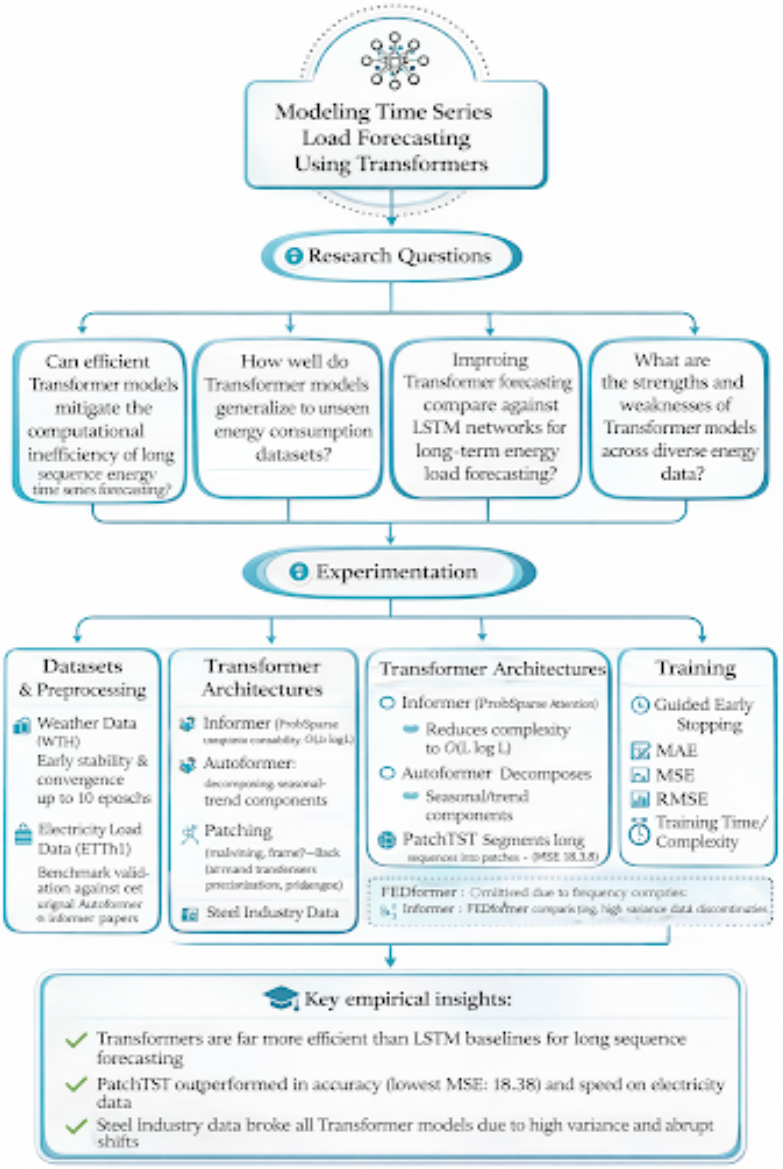


Figure 3.1: Research Methodology Flowchart

3.2 Dataset

This study utilises four publicly available datasets, selected to provide a diverse testing ground ranging from controlled meteorological conditions to volatile industrial energy consumption and high-resolution residential load profiles. This diversity is deliberate: it allows the research questions posed in Chapter 1 — particularly RQ2, concerning generalisation to industrial data — to be tested directly rather than assumed from performance on a single benchmark. Table 3.1 summarises the four datasets before the detailed discussion that follows.

Dataset	Records	Resolution	Features	Role in Study
WTH	35,064	Hourly	12	Calibration / epoch-budget selection
ETTh1	17,420	Hourly	7 (33 engineered)	Continuous control benchmark
STEEL	35,040	15-minute	11(7 retained)	Industrial stress test
Smart Meters (London)	>3.4M (pre-window)	Half-hourly	26 (enriched)	Large-scale residential application

3.2.1 The Weather (WTH) Dataset

The Weather (WTH) dataset contains hourly meteorological records comprising 35,064 total entries across 12 variables, including temperature, humidity, air pressure, wind speed, and visibility. In this research, WTH served as the calibration dataset used to determine a stable training configuration and early-stopping window before committing to more computationally expensive runs on the remaining datasets. Because WTH is smaller and less noisy than the electricity datasets, it allowed rapid iteration: by comparing model convergence behaviour over 5 and 10 training epochs, the study established the epoch budget and early-stopping patience used in subsequent experiments, described further in Section 3.5. The dataset's smooth, continuous, and strongly periodic character — diurnal temperature cycles and seasonal humidity trends being the dominant sources of structure — also makes it a useful sanity check: any architecture that fails to converge stably on WTH would raise concerns about the correctness of the implementation itself, rather than about the architecture's suitability for harder data, and no such failure was observed for any of the seven models evaluated.

3.2.2 The ETTh1 (Electricity Transformer Temperature) Dataset

The ETT family of datasets was derived from electrical power grid transformers monitored in China over a two-year period, and is split into hourly (ETTh1, ETTh2) and 15-minute (ETTm1, ETTm2) variants sharing the same multivariate structure of six load factors and one oil-temperature target [7]. This study uses ETTh1, containing 17,420 hourly

samples in its canonical form, chosen over the minute-level variants because its lower sampling frequency substantially reduces the memory overhead imposed by the quadratic scaling of self-attention with sequence length [1], while still providing 24 data points per day of context for the Lookback-96 window used throughout this thesis.

ETTh1 is used in this research as a canonical benchmark against which the training pipeline's technical correctness can be validated prior to evaluation on more volatile data. Its native structure comprises an hourly timestamp, six exogenous load variables (HUFL, HULL, MUFL, MULL, LUFL, LULL), and a single target variable, Oil Temperature (OT), which the models are trained to forecast 24 steps ahead. As detailed in Section 3.3, this study's feature-engineering pipeline expands the native seven-column structure to 33 input features per timestep through lagged values, rolling statistics, and cyclical calendar encodings; this expansion is disclosed explicitly here because it constitutes a methodological departure from the seven-feature configuration used in the original Informer, Autoformer, and PatchTST publications, a point revisited when this thesis's results are compared against those publications' own reported benchmarks in Chapter 4.

The dataset's strong 24-hour and 7-day periodicity, combined with clear correlation between load variables and the target, makes it well suited to Transformer architectures whose self-attention mechanisms can directly relate observations separated by exactly these periodic intervals. Because ETTh1 is comparatively well-behaved and continuous, it also serves as a control condition: strong performance here establishes a baseline of confidence that is then contrasted against the STEEL dataset described next, where that continuity assumption is deliberately violated.

3.2.3 The Steel Industry Energy Consumption Dataset

To test the robustness of the evaluated architectures under conditions that depart substantially from curated forecasting benchmarks, this study incorporates the Steel Industry Energy Consumption dataset [12]. Unlike WTH and ETTh1, industrial energy data of this kind is characterised by high volatility, abrupt machinery on/off cycles, and complex multivariate interactions rather than smooth periodicity. The dataset comprises 35,040 records at 15-minute resolution across 11 variables, including real-time energy usage (Usage_kWh), lagging and leading reactive power, CO_2 emissions, and a categorical load-type indicator (Light, Medium, or Maximum load); of these, seven continuous

variables were retained for modelling after feature selection.

As documented in the data-loading logs for this study, the raw feature scales in this dataset are markedly heterogeneous: energy usage ranges from 0 to 157.18 kWh (mean 27.39, std 33.44), while the seconds-since-midnight (NSM) feature ranges from 0 to 85,500 (mean 42,750, std 24,940.53) and the CO_2 channel ranges from 0 to only 0.07 (mean 0.01) — a span of roughly six to seven orders of magnitude across the seven variables retained after feature selection. Because industrial machinery can be switched on or off effectively instantaneously, the resulting load profile also exhibits sharp, discontinuous vertical transitions rather than the gradual trends typical of environmental or residential data. This dataset therefore serves in this research as a deliberate stress test, designed to determine whether the architectural advantages reported for Transformer models on curated benchmarks persist under scale heterogeneity and discontinuity of this kind; the results of this stress test, including a documented and mechanistically refined failure analysis, are reported in Chapter 4.

3.2.4 The Smart Meters in London Dataset

To evaluate model performance in a practical, large-scale urban context, this research incorporates the Smart Meters in London dataset, provided by UK Power Networks under the Low Carbon London programme [36]. The dataset contains half-hourly electricity consumption readings for a sample of London households collected between November 2011 and February 2014, enriched in this study's processed form with weather, household, and calendar/holiday covariates, yielding 26 input features. At over 3.4 million raw readings prior to windowing, this is by a wide margin the largest dataset used in this thesis — roughly two orders of magnitude larger than ETTh1 — and it serves as the primary practical-application benchmark for comparing Transformer-based models against the recurrent baselines under realistic residential load conditions, including genuine household-level noise such as localised consumption spikes from individual appliances. The scale of this dataset also has direct methodological consequences documented in Section 3.5: at this volume, a single Transformer training epoch can require close to an hour of wall-clock time on the target hardware, which necessitated an explicit, disclosed epoch cap rather than full convergence to early-stopping criteria for this dataset alone.

3.3 Preprocessing and Feature Engineering

A unified preprocessing and feature-engineering pipeline was applied to all four datasets to ensure that comparisons between architectures reflect genuine model differences rather than inconsistencies in data preparation. Given the specific challenges of time-series data, this pipeline was designed with particular care to preserve chronological structure while addressing missing values, distributional shift, and scale heterogeneity.

Missing values, which constituted a small proportion of the total data in every dataset used, were handled via linear interpolation, preserving the smooth transition of the underlying signal without introducing the step artefacts that simple forward-fill methods can produce. Following this, all continuous features were normalised using Z-score standardisation via scikit-learn's StandardScaler [37]:

$$z = (x - \mu) / \sigma$$

where μ and σ are the mean and standard deviation of each feature computed on the training partition only, and then applied unmodified to the validation and test partitions to avoid information leakage. As discussed in Chapter 4, this standardisation proved sufficient for the WTH, ETTh1, and Smart Meters datasets, but was, on its own, insufficient to prevent numerical instability in two of the three Transformer architectures on the STEEL dataset, a finding examined in detail in the results chapter.

The data was then divided chronologically into training, validation, and test partitions in a ratio of 70% / 20% / 10%, consistent across all four datasets and confirmed directly by the data-loader shapes recorded during experimentation (for example, ETTh1 yields 12,177 training, 3,479 validation, and 1,740 test sequences, an exact 70/20/10 split of the dataset's 17,396 usable windows). This chronological, non-shuffled split is essential for time-series evaluation: because the split preserves temporal order, the model is always evaluated on data strictly later than anything seen during training, preventing look-ahead bias and mirroring the realistic forecasting scenario in which future information is unavailable at training time [43].

Feature engineering was applied to help each model capture temporal structure more directly. Calendar features (hour of day, day of week, month) were encoded using sine–cosine cyclical transformations of the form $\sin(2\pi t/T)$ and $\cos(2\pi t/T)$, where t is the

feature's raw value and T its period (for example, T = 24 for hour-of-day), ensuring that the 23rd hour is represented as numerically adjacent to the 0th hour rather than as a discontinuous jump [4]. For the datasets richest in exogenous structure — ETTh1, STEEL and Smart Meters — lagged values (at 1, 6, 12, and 24-hour intervals) and rolling-window statistics (mean and standard deviation over 3-, 12-, and 24-hour windows) were additionally computed, providing the models with explicit short-term trend and volatility markers rather than requiring these to be inferred purely from raw sequences. This engineering step is what expands ETTh1 from its native 7 columns to the 33 input features confirmed in the data-loader logs, and is disclosed here as a specific, quantified methodological choice rather than an implicit detail.

All datasets were then arranged into fixed-length input–output windows using a Lookback of 96 timesteps, a Label length of 48 timesteps, and a prediction Horizon of 24 timesteps, consistent across every model and dataset in this study and confirmed directly in the corresponding data-loader initialisation logs. The Label length overlaps with the final 48 steps of the Lookback window and is provided to the Transformer decoders as a “start token” containing recent ground-truth context, following the generative decoding scheme used by Informer and Autoformer, while the recurrent models and PatchTST consume only the Lookback window directly.

3.4 Comparative Model Architectures and Core Mechanics

This research evaluates two families of sequence-modelling architecture — Transformer-based models and Recurrent Neural Networks (RNNs) — which differ fundamentally in how they process temporal information. Transformers process the entire input sequence in parallel via self-attention, computing relationships between arbitrary pairs of timesteps directly; RNNs instead process the sequence step by step, propagating information forward through an evolving hidden state [1, 5]. For every architecture discussed in this section, a structural diagram is provided to make its internal data flow explicit, in keeping with the presentation style of the original thesis on which this chapter is based.

3.4.1 Transformer-Based Models

The three Transformer variants evaluated in this study — Informer, Autoformer, and PatchTST — each adapt the standard multi-head self-attention meta-architecture to address its quadratic time and memory complexity with respect to sequence length, while pursuing different strategies for capturing time-series-specific structure. Before examining each variant individually, it is useful to recall the general Transformer meta-architecture from which all three descend.

The Standard Transformer Meta-Architecture

The vanilla Transformer, illustrated in Figure 3.1, follows an encoder–decoder structure in which both encoder and decoder are composed of Nx identical stacked blocks [1]. Each encoder block consists of a multi-head self-attention module followed by a position-wise feed-forward network, with residual ("Add") connections and layer normalisation applied around each sub-layer; each decoder block additionally inserts a masked multi-head self-attention module and a cross-attention module that attends over the encoder's output. Positional encoding is added to both the input and output embeddings, since the architecture itself contains no recurrence and would otherwise be blind to sequence order. The output of the final decoder block is projected through a linear layer and, in the original language-modelling formulation, a softmax to produce output probabilities; in the time-series forecasting adaptations used in this thesis, this final projection instead produces continuous-valued forecasts directly.

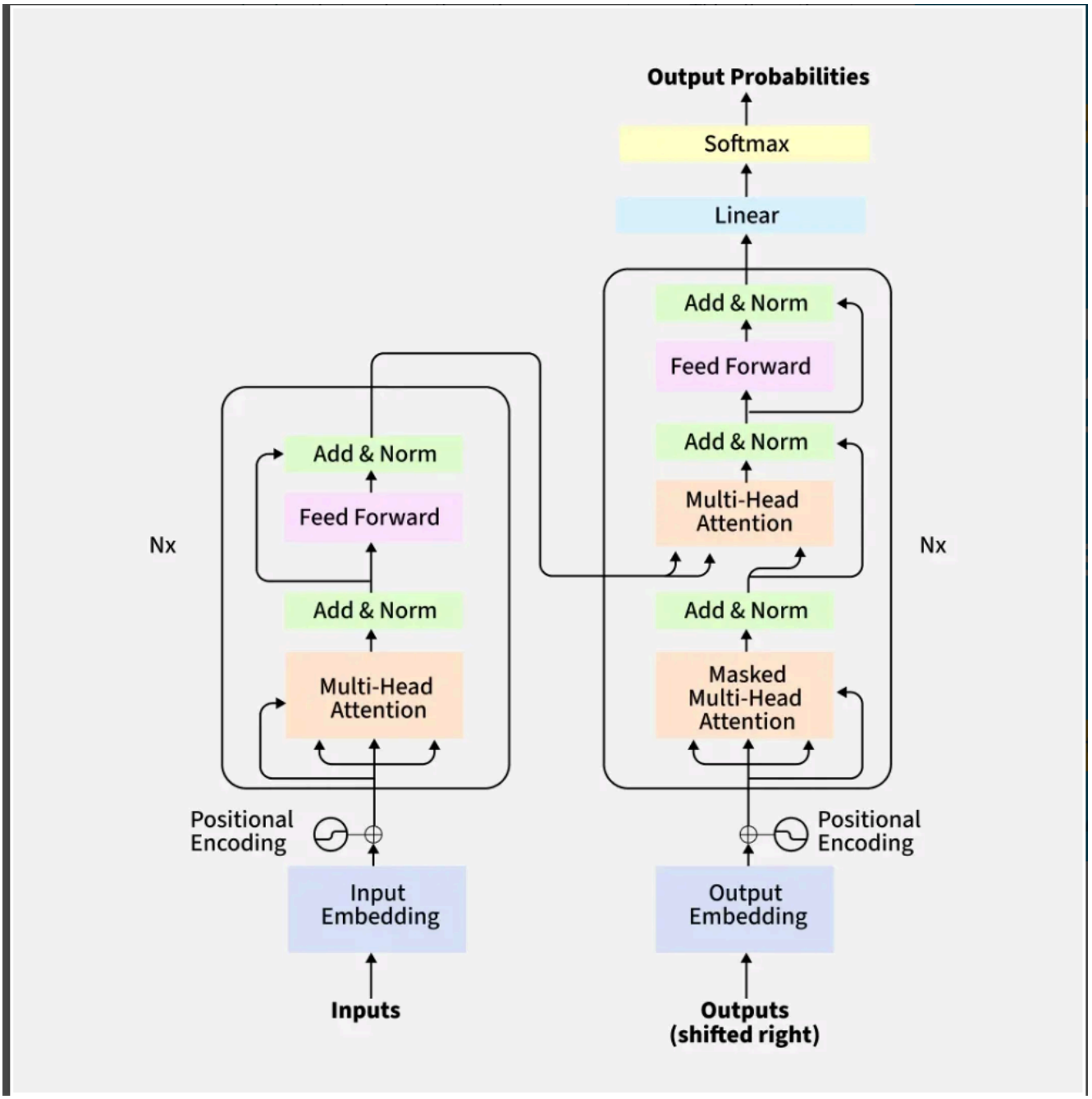


Figure 3.2: Standard Transformer Encoder–Decoder Architecture

The core computational primitive underlying every block is scaled dot-product attention over a Query-Key-Value (QKV) representation:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

where Q, K, and V are learned linear projections of the input, and d_k is the dimensionality of the key vectors, used to rescale the dot product before the softmax to avoid vanishingly small gradients. Multi-head attention computes several such attention functions in parallel over different learned projections of the input, allowing the model to attend to different representation subspaces simultaneously. It is this mechanism's $O(L^2)$ time and memory cost with respect to sequence length L that motivated the three

efficiency-oriented variants examined next.

Informer: ProbSparse Attention and Self-Attention Distilling

The Informer model represents a pivotal advancement in the Long-term Time Series Forecasting (LSTF) landscape, specifically designed to mitigate the $O(L^2)$ time complexity and memory usage of the vanilla Transformer. Its architecture is built upon the observation that the distribution of self-attention is often sparse, meaning only a few dot-product pairs contribute significantly to the overall attention mechanism. To exploit this, Informer introduces the ProbSparse Self-Attention mechanism. This technique utilizes a "Query-Sparsity" measurement to identify the most dominant queries, allowing the model to compute attention only for these high-probability pairs while ignoring the "lazy" or redundant ones. This mathematical refinement successfully reduces the computational burden to O(L log L)[7] enabling the model to handle much longer lookback windows without exhausting hardware resource

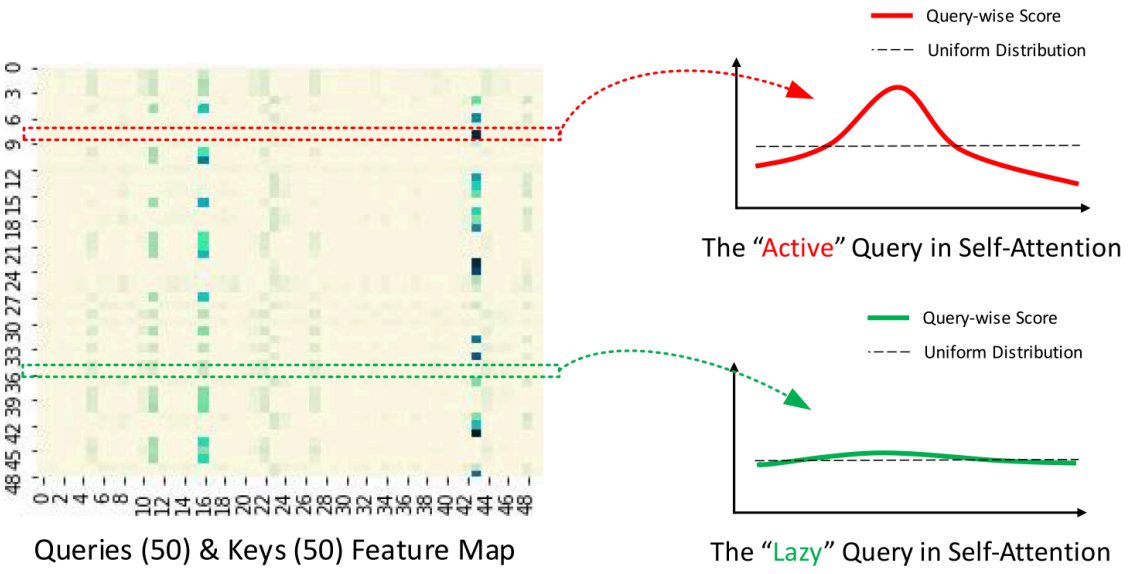


Figure 3.3: ProbSparse Attention Mechanism

Beyond its attention mechanism, the Informer utilizes a specialized encoder-decoder structure to streamline the forecasting process. The encoder implements Self-Attention Distilling, a process that applies 1D convolutions and max-pooling between attention layers. This "distills" the hidden state by halving its temporal dimension at each step, ensuring the model focuses only on the most critical temporal features while maintaining a manageable memory footprint. On the receiving end, the decoder is designed for Generative

Inference, which stands in stark contrast to the recursive, step-by-step prediction methods of traditional RNNs.

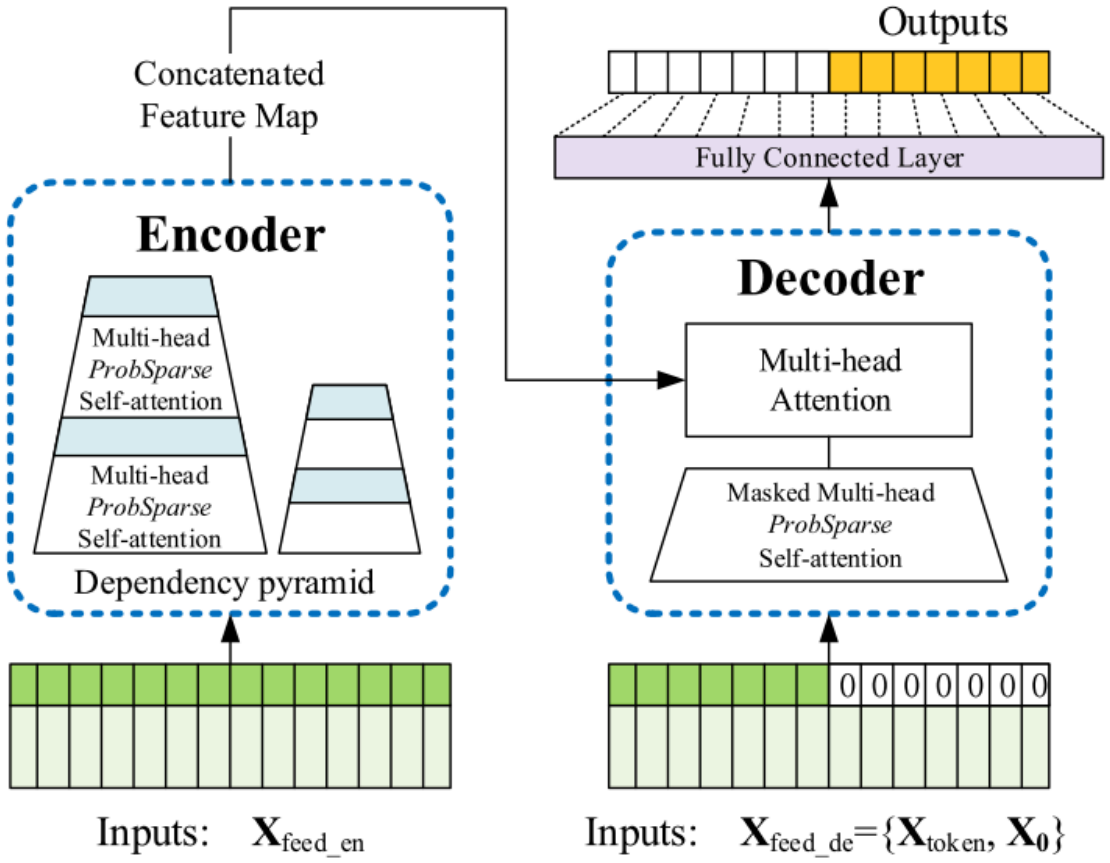


Figure 3.4: Informer Architecture

In this study's implementation, Informer's parameter count varied by dataset according to the model-dimension settings feasible within the 6GB VRAM budget, ranging from approximately 2.90 million parameters on WTH and ETTh1 down to 87,623 parameters under the deliberately reduced-capacity configuration used for STEEL and 27,610 parameters under the memory-optimised configuration used for the much larger Smart Meters dataset.

Autoformer: Series Decomposition and Auto-Correlation

The Autoformer architecture represents a paradigm shift in long-term forecasting by moving beyond simple attention mechanisms to a "decomposition-based" approach. Traditional Transformers often struggle with time series because they treat the sequence as a flat set of tokens, ignoring the underlying structural components like long-term trends and seasonal fluctuations. Autoformer addresses this by embedding a Series Decomposition Block directly into its architecture. This block continuously separates the input hidden

states into two distinct paths: the Trend-Cyclical component, which captures the long-term progression of the data, and the Seasonal component, which captures the repeating, high-frequency patterns. This internal separation allows the model to handle the inherent non-stationarity of energy datasets like ETTh1 and Smart London more effectively than vanilla architectures.

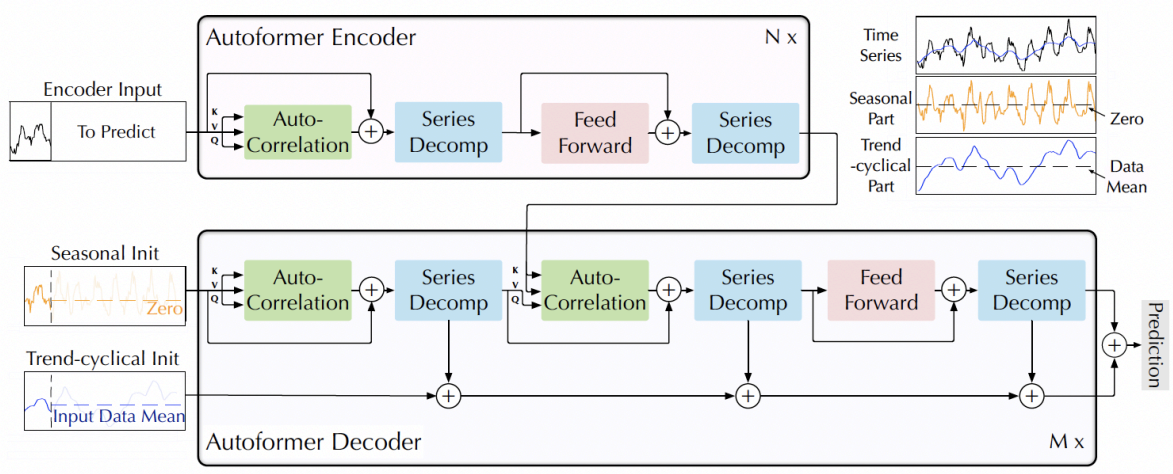


Figure 3.5: Autoformer Architecture with Series Decomposition

The second core innovation of the Autoformer is its replacement of the standard dot-product self-attention with the Auto-Correlation Mechanism. While standard attention calculates point-to-point similarities—a process that is both computationally expensive and susceptible to noise—Auto-Correlation identifies sub-series level similarities. It utilizes the Fast Fourier Transform (FFT) to calculate autocorrelation in the frequency domain, allowing the model to discover period-based dependencies. By aggregating similar sub-series segments across different periods, the Auto-Correlation mechanism provides a much stronger inductive bias for time series data. This mechanism reduces the computational complexity to O(L log L) [8] while significantly improving the model's ability to capture the multi-seasonal rhythms found in residential and industrial load profiles.

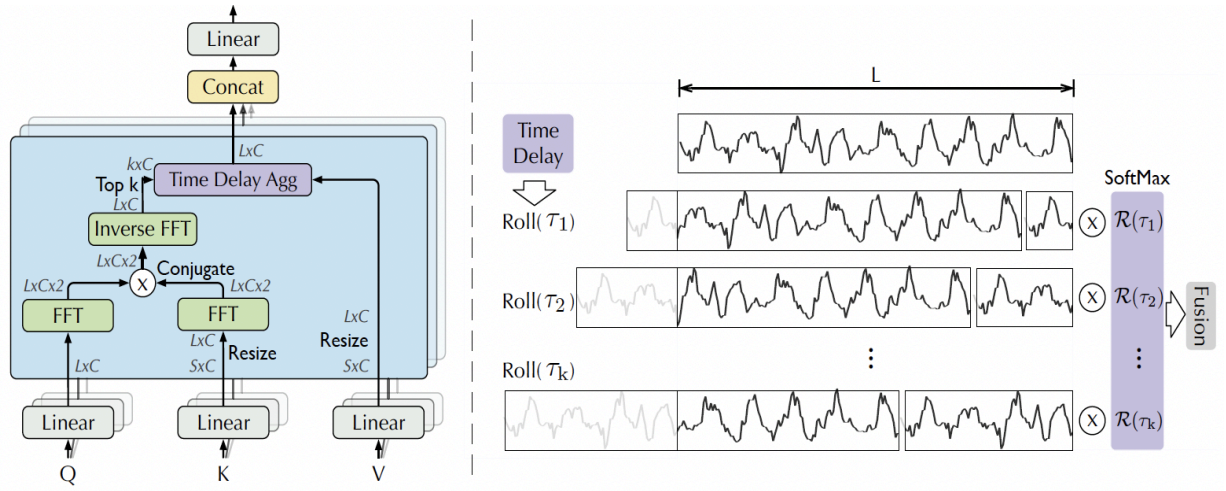


Figure 3.6: Auto-Correlation Mechanism and Period-Based Dependencies

In our experimental results, the Autoformer demonstrated a high degree of predictive stability, particularly in capturing the cyclical trends of the ETTh1 dataset. However, this structural complexity comes with a trade-off in efficiency. As illustrated in our comparative training time analysis, the multiple decomposition stages and the frequency-domain calculations resulted in the longest training duration among all tested models, exceeding 68,000 seconds. While the Autoformer is an exceptionally powerful tool for capturing complex, repeating patterns, its deployment in resource-constrained or real-time environments requires significant computational overhead compared to more streamlined models like PatchTST.

PatchTST: Sub-series Patching and Channel-Independent Processing

PatchTST (Patch Time Series Transformer) represents the most significant departure from the point-wise processing paradigm found in earlier Transformer models. While the Informer and Autoformer treat individual time steps as tokens, PatchTST introduces the concept of Sub-series Patching. In this approach, the input sequence is segmented into overlapping or non-overlapping patches—for example, grouping 16 consecutive time steps into a single patch with a stride of 8. This method effectively transforms the input from a long, thin vector into a series of information-dense tokens. By focusing on patches rather than points, the model can capture local semantic patterns that are often lost in the noise of individual observations, while simultaneously reducing the attention complexity from O(L^2) to O (N^2), where (N)is the significantly smaller number of patches.

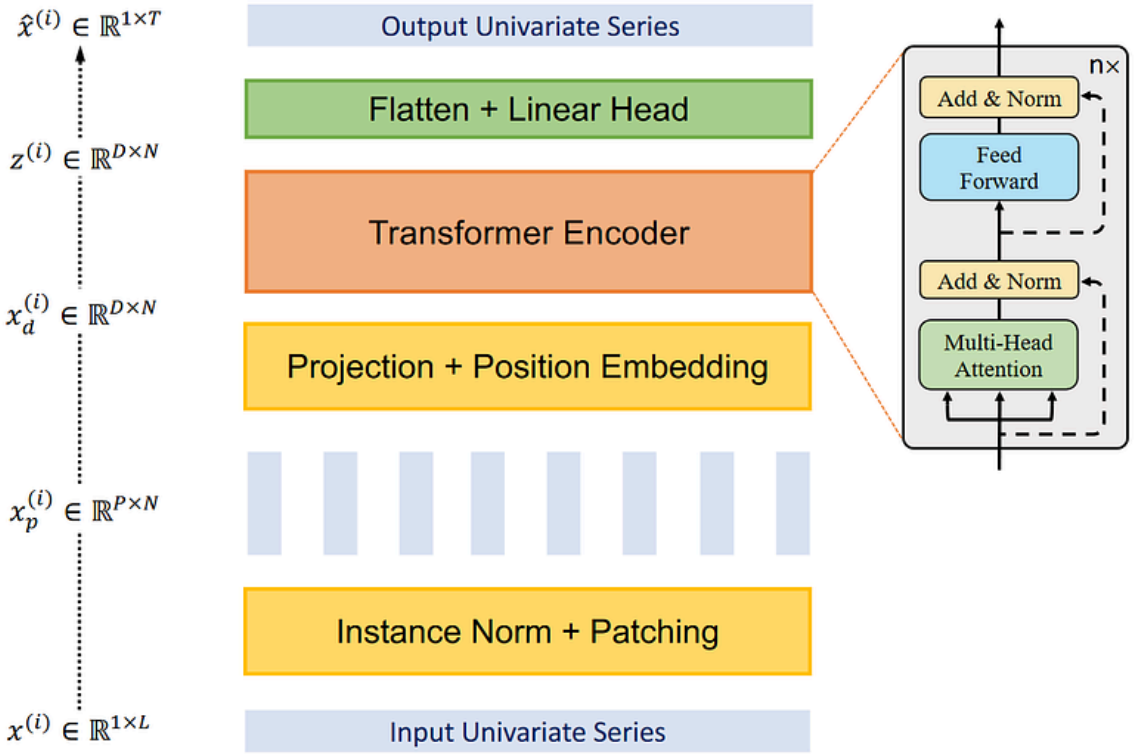


Figure 3.7: Sub-series Patching and Sequence Segmentation

Complementing the patching layer is the principle of Channel Independence. In many multivariate time-series tasks, different sensors or variables (such as Temperature vs. Electricity Load) may have vastly different scales or patterns. Traditional models often attempt to flatten these into a single embedding, which can lead to feature interference. PatchTST instead processes each variable as an independent channel. Each channel is fed through the same Transformer backbone with shared weights, which allows the model to learn universal temporal patterns while maintaining the unique signal integrity of each specific variable. This independence is particularly crucial for the Smart London dataset, where residential anomalies in one household's consumption should not be allowed to corrupt the broader community load profile.

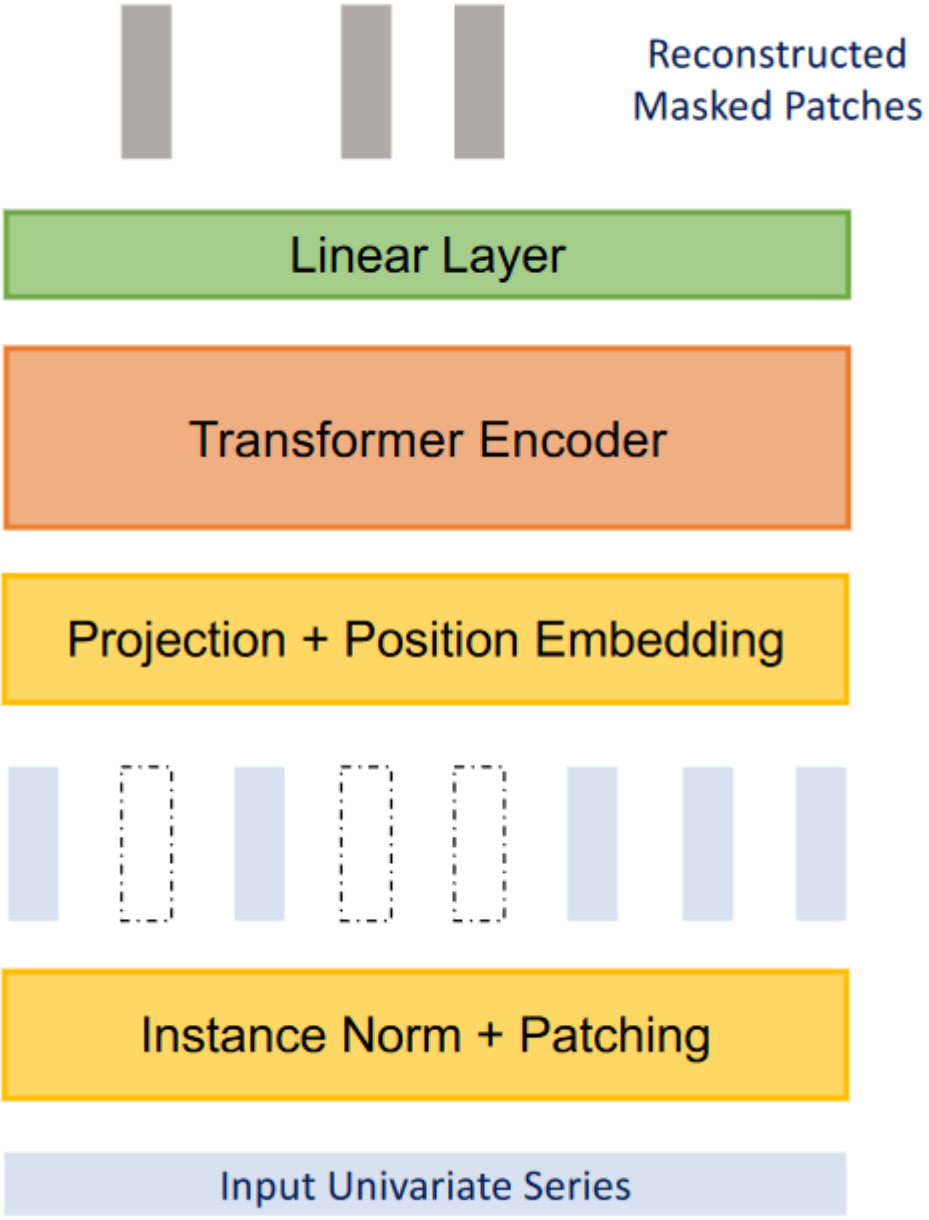


Figure 3.8: Patching and Temporal Feature Extraction

The overall architecture of PatchTST utilizes a standard Transformer Encoder but applies it to these extracted patches. Following the encoding process, a simple linear head is used for the Prediction Block, mapping the patch representations directly to the target Horizon 24. This design proved to be the most efficient in our research; because the effective sequence length is reduced by the patching factor, the model consumes far less VRAM and trains significantly faster than the Autoformer. As a result, PatchTST achieved the best performance in our empirical analysis, producing an MSE of 18.38 and demonstrating that "less is more" when it comes to token granularity in long-term

forecasting.

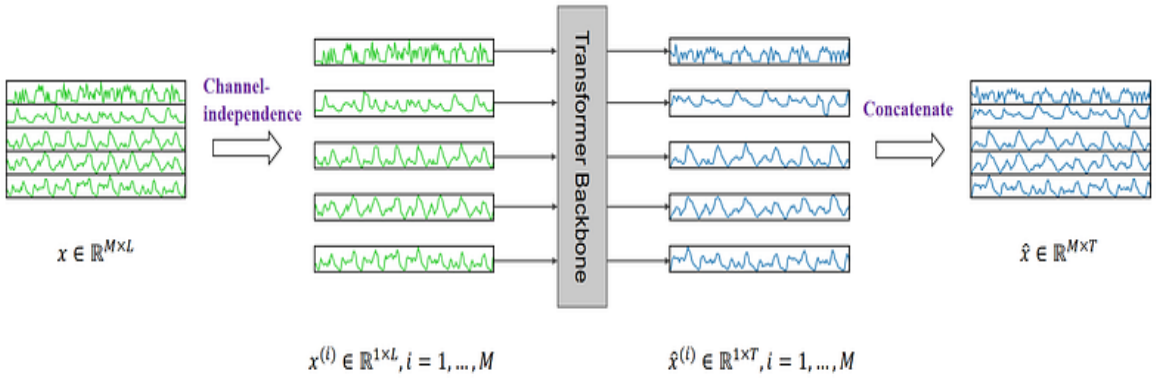


Figure 3.9: PatchTST Backbone and Prediction Head

These models, based on the Transformer architecture, shift the paradigm toward better efficiency compared to the earlier model versions, while the achieved predictive accuracy remains at a comparable level. The capability of parallel processing for long sequences and handling large data with minimal computational overhead is an important advantage in performing forecasting tasks under resource-constrained conditions.

3.4.2 Recurrent Neural Network Baselines

In contrast to the parallel processing of Transformer architectures, the four recurrent baselines evaluated in this study process sequences step by step, maintaining an internal hidden state that carries information forward through time [5]. Figure 3.4 illustrates the basic recurrent computational unit common to this entire family: at each timestep t, the current input x□ is combined with the hidden state carried forward from the previous timestep and passed through a nonlinearity — typically tanh — to produce the updated hidden state h□, which is both output and propagated forward to the next timestep.

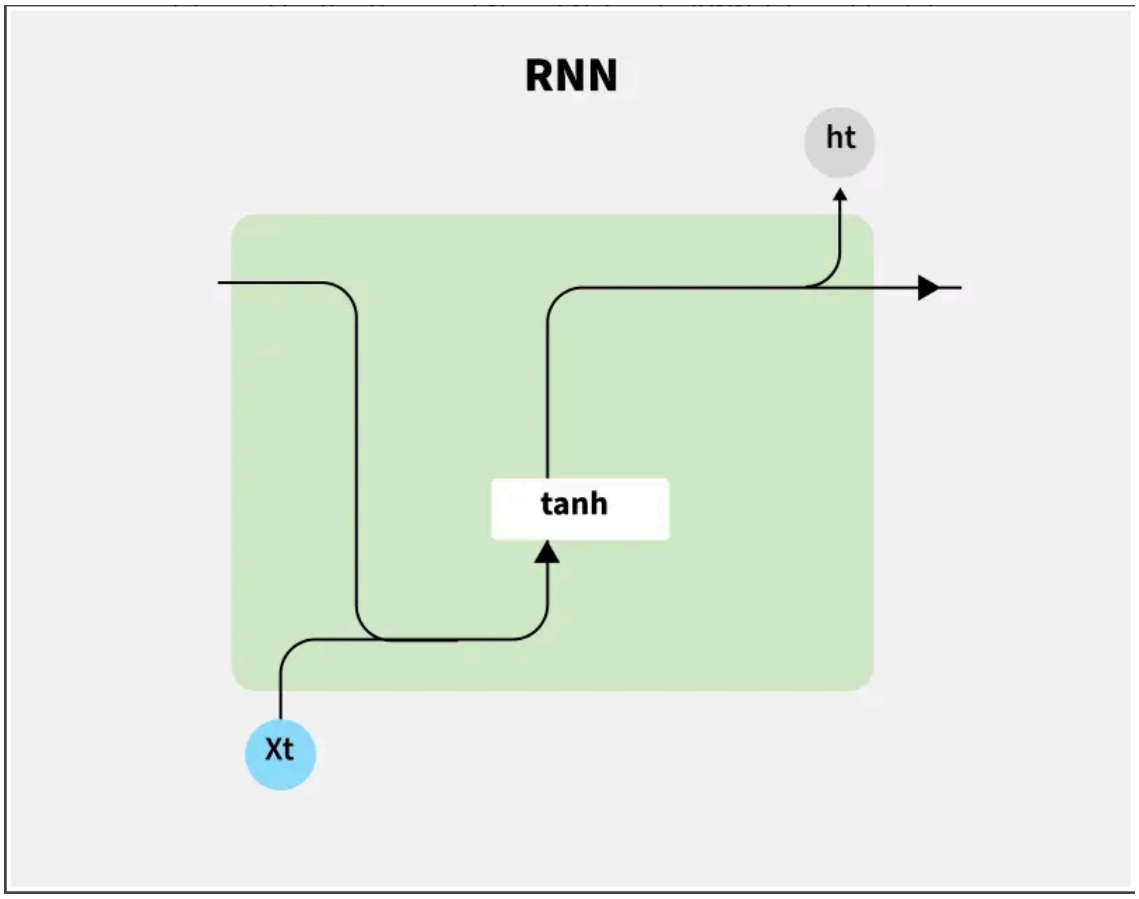


Figure 3.10: Basic Recurrent Neural Network Unit

A plain RNN of this form is prone to the vanishing and exploding gradient problem when trained via backpropagation through time over long sequences, because the same weight matrix is repeatedly multiplied through the unrolled computational graph. This limitation, well documented in the literature [41], is the direct motivation for the gated architectures described in the remainder of this section.

LSTM (Long Short-Term Memory)

The LSTM addresses the vanishing-gradient problem through a gated memory-cell design comprising three gates, each implemented as a sigmoid-activated linear layer over the concatenation of the current input and previous hidden state: a Forget Gate, which determines what proportion of the existing cell state to retain; an Input Gate, which determines what proportion of a candidate update (computed via a tanh layer) to add to the cell state; and an Output Gate, which determines what proportion of the (tanh-activated) cell state to expose as the new hidden state [5]. Figure 3.5 illustrates this internal gating structure.

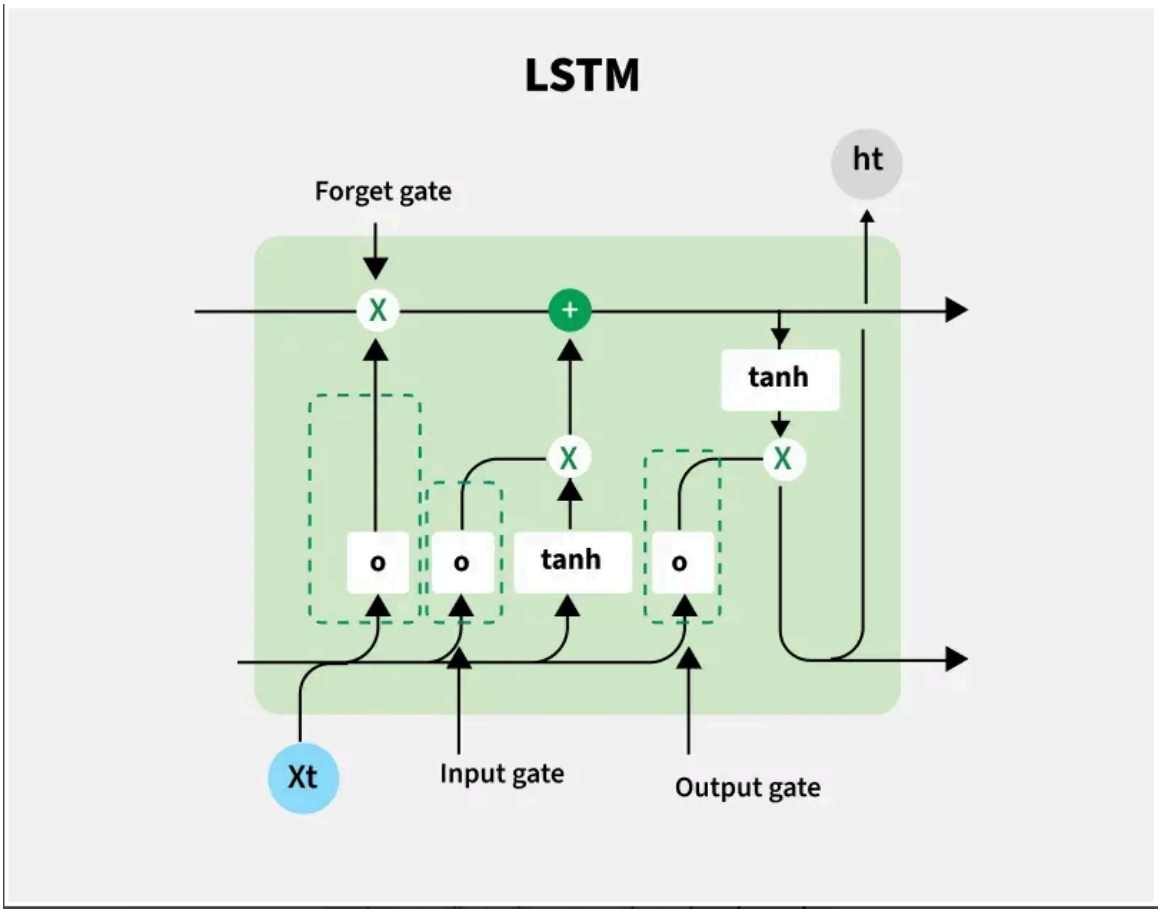


Figure 3.11: LSTM Cell and Gating Structure

A defining property of this gating structure, directly relevant to the failure-mode analysis in Chapter 4, is that every gate is sigmoid-activated (bounding its output to the range [0, 1]) and every content update is tanh-activated (bounding its output to [−1, 1]). This means that, regardless of the magnitude of any single input value, its contribution to the cell state and hidden state at each timestep is mathematically bounded. This sophisticated mechanism enables the LSTM to learn the medium-to-long-range dependencies essential for tracking the daily load cycles in datasets like ETTh1, and it remains a widely used and computationally efficient baseline for load forecasting [6].

BiLSTM (Bidirectional LSTM)

The BiLSTM extends the standard LSTM by processing the input sequence in both temporal directions using two parallel LSTM layers — a Forward Layer processing the sequence from start to end, and a Backward Layer processing it from end to start — and concatenating their hidden states at each timestep, as illustrated in Figure 3.6. This provides the model with both past and future context within the input window at every point in the sequence, which can improve representational richness for dependencies that are not purely

causal.

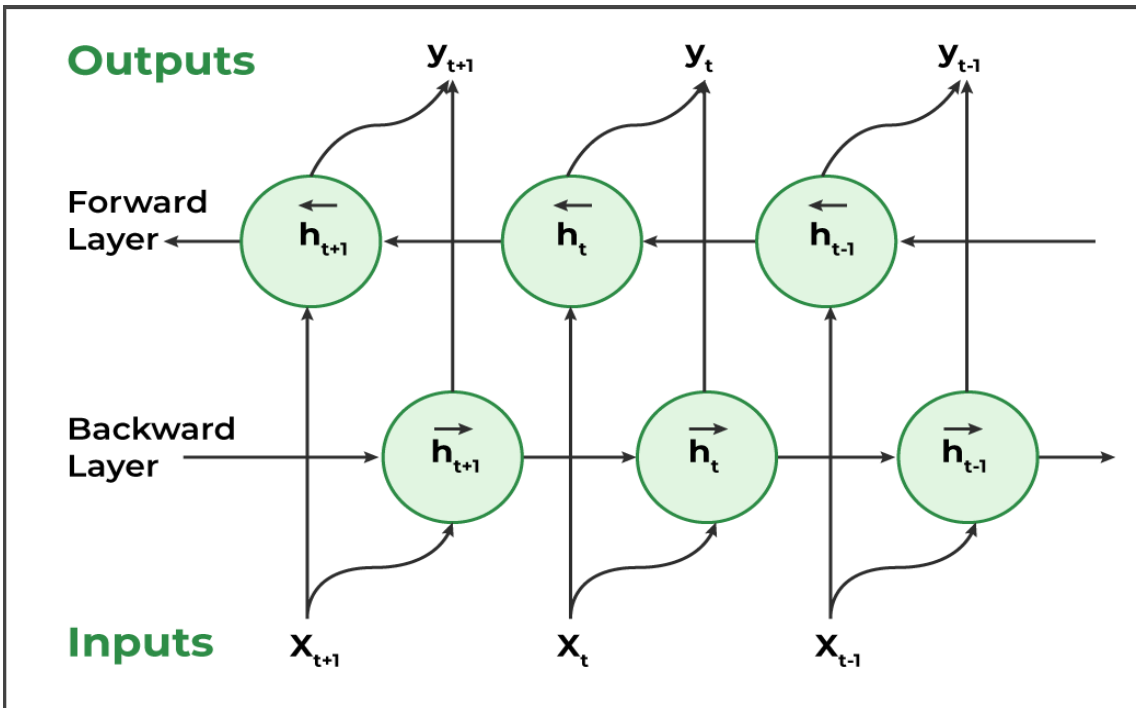


Figure 3.12: Bidirectional LSTM Information Flow

This richer representation typically comes at approximately double the parameter count and training time relative to a unidirectional LSTM of the same hidden size, since two full sets of gating parameters must be learned and both directions' computations must be performed at every timestep. This trade-off is examined empirically in Chapter 4, where BiLSTM's accuracy relative to plain LSTM varies by dataset.

GRU (Gated Recurrent Unit)GRU (Gated Recurrent Unit)

The GRU simplifies the LSTM's three-gate structure into two gates — a Reset Gate and an Update Gate — and merges the cell state and hidden state into a single vector, as shown in Figure 3.7. The Reset Gate determines how much of the previous hidden state to disregard when computing a candidate update (via a tanh layer), while the Update Gate determines the interpolation between the previous hidden state and this candidate to produce the new hidden state, implemented through the (1 − z) and z weighting shown in the diagram.

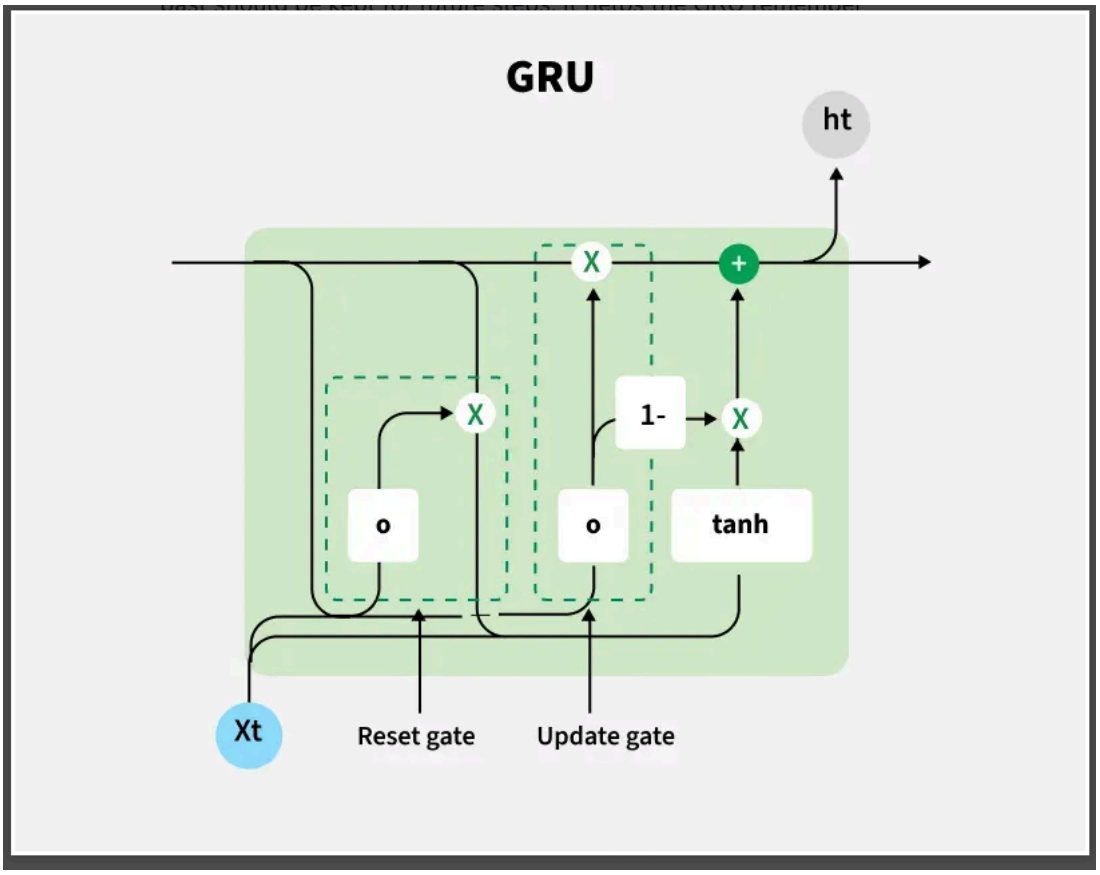


Figure 3.13: GRU Cell and Gating Structure

Like the LSTM, every gate in the GRU is sigmoid-activated and every content update is tanh-activated, preserving the same bounded-activation property discussed above. Because it has fewer parameters than the LSTM, the GRU is computationally lighter and often converges faster, making it particularly well suited to scenarios where training efficiency is a priority; as shown in Chapter 4, GRU offered a strong accuracy-per-parameter trade-off across all four datasets evaluated in this study, and was in some cases the single most accurate recurrent model tested.

The Minimal Single-Layer LSTM Configuration

To establish an explicit lower bound on model capacity within the recurrent family, this study additionally evaluates a deliberately minimal, single-layer LSTM configuration with a hidden dimension of 16 units — substantially smaller than the 32–64-unit hidden dimensions typically used for the LSTM and GRU baselines above — whose final hidden state is projected directly to the output horizon without additional intermediate layers. This configuration, denoted EfficientLSTM in the accompanying experiment logs and referred to as LSTM-16 in Chapters 4 and 5, shares the identical internal gating structure illustrated in Figure 3.5 and is treated in this thesis as a capacity-reduced variant of the standard LSTM

architecture rather than a novel design; its role is specifically to quantify the accuracy sacrificed at the smallest practical parameter footprint. Across the datasets evaluated, its parameter count ranged from 4,456 (STEEL) to 13,424 (Smart Meters), consistently the smallest of any model tested. As reported in Chapter 4, this configuration was competitive with the other three recurrent baselines on the smaller, seven-feature STEEL dataset, but was the least accurate model of all seven evaluated on the larger, twenty-six-feature Smart Meters dataset — direct evidence that a fixed minimal capacity is adequate for some forecasting tasks and inadequate for others, rather than being unconditionally sufficient or insufficient.

3.5 Training Approaches

The training strategy was designed to enable a fair, reproducible comparison of all seven models while remaining feasible within the 6GB VRAM budget described in Section 3.1. Training proceeded in two stages: an initial calibration stage on the WTH dataset to establish a stable epoch budget, followed by a final training stage applied consistently across all four datasets.

3.5.1 Initial Calibration on the WTH Dataset

The first stage of training was conducted on the WTH dataset specifically because its smaller size allowed rapid iteration. All models were trained separately, and their validation-loss trajectories over 5 and 10 epochs were compared to determine an appropriate epoch budget and early-stopping patience. This calibration surfaced an important methodological finding, detailed further in Chapter 4: at 5 epochs, Informer achieved better summary error statistics than Autoformer despite producing visibly flatter, less temporally faithful predictions, indicating that a short training budget alone was insufficient to distinguish genuine pattern learning from a low-error but uninformative mean-reverting prediction. This motivated the adoption of a 10-epoch training budget, with early-stopping patience of 5–7 epochs depending on model family, for all subsequent experiments.

3.5.2 Final Training Across Datasets

With the epoch budget established, all seven models were trained on each of the four datasets under a matched protocol. The Adam optimiser [38] was used throughout, with a

learning rate of 1×10^{-4} for the three Transformer models and 1×10^{-3} for the four recurrent models, values chosen to reflect the differing training stability characteristics of each family [39]. Early stopping monitored validation loss with a patience of 5 epochs for the recurrent models and 7 epochs for the Transformer models [40], halting training once no further improvement was observed and thereby avoiding unnecessary computation once a model had converged or, in the case of the STEEL dataset's Informer and Autoformer runs, once no meaningful validation signal was present at all.

Batch size was set to 32 for all four recurrent models across every dataset. For the Transformer models, batch size was constrained by the 6GB VRAM budget and varied by dataset according to model size and input dimensionality: a batch size of 8 was used for the Transformer models on WTH, ETTh1, and STEEL, while a batch size of 16 was used on the substantially smaller-capacity Transformer configurations trained on the Smart Meters dataset. This dataset-dependent adjustment is reported here explicitly, in line with the hardware-aware framing of this thesis, rather than treated as a hidden methodological detail: it reflects a genuine, disclosed consequence of training under a fixed memory budget rather than an inconsistency in experimental design.

3.5.3 Model-Specific Training Considerations

Each architecture imposed distinct practical constraints on training. The Transformer models' memory consumption scales with the product of sequence length and model dimension due to the self-attention mechanism, which required their internal dimensions to be reduced substantially on the STEEL and Smart Meters datasets relative to WTH and ETTh1 in order to remain within the available VRAM; PatchTST's patch-based tokenisation made it consistently the least memory-intensive of the three Transformers under every configuration tested. The recurrent models required comparatively little adjustment across datasets, with hidden dimensions of 16–64 units sufficient in every case, reflecting their more modest, linear-in-sequence-length memory footprint.

Training duration and convergence behaviour also differed substantially between families. Across all datasets on which training was stable, the Transformer models required considerably more wall-clock time than the recurrent baselines to complete an equivalent number of epochs — for instance, on the Smart Meters dataset, Informer and Autoformer required 11,119.1 and 10,694.7 seconds respectively to complete three epochs, against

1,189.7–1,592.2 seconds for the four recurrent models to complete the same three epochs, a difference of roughly one order of magnitude attributable primarily to the larger per-batch attention computation and the smaller batch sizes necessitated by VRAM constraints. Each model's final evaluation was conducted on the held-out test partition only after its validation loss had stabilised or its early-stopping criterion had triggered, and training time, parameter count, and peak memory occupancy were recorded alongside predictive accuracy for every run, consistent with the joint accuracy–efficiency evaluation framework introduced in Chapter 1.

3.6 Evaluation Metrics

Consistent with the joint accuracy–efficiency framework motivating this thesis, model evaluation is bifurcated into statistical accuracy metrics and computational efficiency benchmarks; neither is treated as sufficient on its own to characterise a model's practical suitability for deployment.

3.6.1 Statistical Accuracy Metrics

Given a set of n test-set predictions $\hat{y}_i$ and corresponding ground-truth values y_i, the following three metrics are computed:

- Mean Absolute Error (MAE) = $(1/n) \Sigma |y_i - \hat{y}_i|$: the average magnitude of prediction error in the original units of the target variable, providing a directly interpretable measure of typical deviation [42].
- Mean Squared Error (MSE) = $(1/n) \Sigma (y_i - \hat{y}_i)^2$: the average squared prediction error, which penalises large deviations disproportionately and is therefore sensitive to the occasional large errors that matter most in operational load-forecasting contexts [42].
- Root Mean Squared Error (RMSE) = $\sqrt{MSE}$: the square root of MSE, returning the error to the original units of the target variable while retaining sensitivity to large deviations.

Mean Absolute Percentage Error (MAPE) was also computed during experimentation but is reported only where meaningful: on the Smart Meters dataset, MAPE returned an infinite or undefined value for every model tested, a data artefact caused by exact zero values in the target series triggering division by zero, and is therefore excluded from the

comparative tables in Chapter 4 in favour of the three metrics above.

3.6.2 Computational and Operational Metrics

- Training Time: total wall-clock time in seconds to reach the early-stopping criterion or the epoch budget, used to assess the practical cost of achieving a given level of accuracy on the target hardware.
- Parameter Count: the total number of trainable parameters in each model configuration, used as a hardware-independent measure of model complexity and memory footprint.
- GPU Memory Usage (VRAM): peak memory occupancy during training, directly relevant given the 6GB VRAM budget of the hardware used throughout this study, and the metric most responsible for the dataset-dependent batch-size and model-dimension adjustments described in Section 3.5.

3.7 Experimental Setups

All experiments were executed on a single ASUS ROG Zephyrus G14 (GA401IV) laptop with an NVIDIA GeForce RTX 2060 (Max-Q, 6GB VRAM), using PyTorch as the underlying deep learning framework. Each of the seven models — Informer, Autoformer, PatchTST, LSTM, BiLSTM, GRU, and the minimal single-layer LSTM configuration — was trained and evaluated independently on each of the four datasets described in Section 3.2, following the protocol described in Section 3.5: a chronological 70/20/10 train/validation/test split, a fixed Lookback of 96, Label length of 48, and Horizon of 24, Adam optimisation with learning rates of 1×10^{-4} (Transformers) and 1×10^{-3} (RNNs), and early stopping with patience of 5–7 epochs depending on model family.

Four experimental runs are reported in this thesis: a three-model comparison (Informer, Autoformer, PatchTST) on WTH and on ETTh1; and a full seven-model comparison, including all four recurrent baselines, on both the STEEL and Smart Meters datasets. This staged approach reflects the practical reality of conducting large-scale comparative experiments on constrained, single-GPU hardware, where Transformer training on the largest dataset (Smart Meters) alone required over three hours per model; it is disclosed here explicitly, and the specific dataset–model combinations completed at the time of writing are documented precisely in Chapter 4, alongside an explicit account of

which comparisons remain outstanding.

3.8 Conclusion of Methodological Approach

IThis methodology was designed to bridge the gap between architectural innovation and practical deployment feasibility. By subjecting seven models — from the high-capacity Autoformer to the deliberately minimal single-layer LSTM configuration — to identical preprocessing, a matched (though, where necessary, explicitly disclosed and dataset-conditional) training protocol, and a consistent joint accuracy–efficiency evaluation framework across four datasets of varying statistical regularity, this study provides a transparent and reproducible basis for the model-selection guidance developed in later chapters.

Critically, this methodology does not treat hardware constraints or data irregularity as sources of noise to be minimised, but as first-class experimental variables in their own right. By explicitly recording and reporting parameter counts, training times, and VRAM occupancy alongside predictive accuracy for every model and dataset combination, and by including the STEEL dataset specifically to test the limits of the standard LTSF pipeline under conditions of extreme scale heterogeneity, this research is positioned to explain not only which architecture performs best, but under what data and hardware conditions that ranking holds — and where it breaks down entirely. The results of applying this methodology are presented in full in Chapter 4.

Chapter Four
Experimental Results and Discussion

This chapter presents the implementation, evaluation, and comparative analysis of Transformer-based architectures against recurrent models for long-term time series forecasting (LTSF). The following sections detail the experimental findings across four diverse datasets, culminating in a complete, seven-model comparison on two of them. The primary objective was to assess not only predictive accuracy but also robustness under varying data conditions, computational efficiency, and suitability for deployment in resource-constrained environments.

By integrating quantitative metrics with qualitative pattern analysis and hardware-aware constraints, this chapter offers a holistic benchmark of contemporary forecasting architectures — Informer, Autoformer, and PatchTST — placed in context against four recurrent baselines: LSTM, BiLSTM, GRU, and a deliberately minimal single-layer LSTM configuration. Where earlier stages of this research supported only a provisional or partial account of certain results, this chapter presents the complete, corrected findings obtained once every planned experimental run had concluded, including two results that materially revise conclusions drawn earlier in this project: a refined, mechanistic explanation for the STEEL dataset failure, and a demonstration that PatchTST's advantage over the other Transformers is dataset-conditional rather than absolute.

4.1 Dataset Description

The experiments utilised four publicly available datasets to ensure generalisability across domains with varying scale, sparsity, and temporal pattern. All experiments employed a Lookback of 96 timesteps (historical window), a Label length of 48 timesteps, and a Horizon of 24 timesteps (prediction window) to capture critical daily and sub-daily cycles, as detailed in Chapter 3.

- Weather (WTH): 35,064 hourly meteorological records, used primarily for initial hyperparameter calibration and establishing the early-stopping window.
- ETTh1 (Electricity Transformer Temperature): 17,420 hourly records, an industrial

benchmark featuring Oil Temperature (OT) as the target variable, exhibiting strong 24-hour and 7-day periodicity; used to validate the technical integrity of the training pipeline against established literature.

- STEEL Industry Energy Consumption: 35,040 records at 15-minute resolution, characterised by extreme scale heterogeneity, machinery on/off cycles, and abrupt step-like transitions; used as a stress test for model robustness.
- Smart Meters in London: over 3.4 million half-hourly residential readings prior to windowing, used to evaluate all seven models together in a practical, large-scale urban context.

All datasets followed a consistent chronological 70% / 20% / 10% training, validation, and test split, confirmed directly from the data-loader shapes recorded during experimentation (for example, ETTh1 yields exactly 12,177 / 3,479 / 1,740 sequences, a precise 70/20/10 division), preventing look-ahead bias and ensuring realistic evaluation.

4.2 Preprocessing and Feature Engineering

A unified preprocessing pipeline was applied to all datasets to ensure methodological consistency and enable a fair comparison between Transformer-based and RNN-based architectures. The process began with the maintenance of temporal integrity; missing values—constituting less than 0.5% of the total data—were handled via linear interpolation to preserve the smooth transition of signals without introducing the "step" artifacts often caused by simple fill methods. To achieve numerical homogeneity across variables with varying scales (such as temperature in Celsius vs. energy load in kW), all continuous features underwent Z-score normalization using StandardScaler. This transformation aligned the distributions to a mean of zero and unit variance, a step that proved indispensable for stabilizing gradient flow and preventing scale-induced collapse in the attention-based models.

Feature engineering further enriched the models by capturing complex temporal dynamics. Temporal features were encoded using sine-cosine cyclical transformations, which translate calendar markers (hour, day, month) into a continuous circular space, allowing the models to recognize that the 23rd hour is adjacent to the 0th hour.

Additionally, the pipeline integrated lag features (at 1, 6, 12, and 24-hour intervals) and rolling statistics, such as moving means and standard deviations. These additions provided the models with explicit markers for local trends and volatility, significantly improving prediction stability in high-variance regimes like the STEEL dataset.

4.2.1 PatchTST Mechanics: Sub-series Patching

A core innovation focused on in this research is the Sub-series Patching mechanism utilized by PatchTST. Unlike earlier models like Informer, which treat individual time steps as isolated tokens, PatchTST segments the input sequence of length L into smaller, overlapping patches of length P using a defined stride S. For instance, by grouping 16 time steps into a single patch with a stride of 4, the model transforms a high-resolution sequence into a series of information-dense tokens.

This segmentation process offers two primary advantages. First, it retains local semantic information—the "shape" of a trend—that is often lost in the point-wise noise of standard observations. Second, it mathematically reduces the computational complexity of the self-attention mechanism from $O(L^2)$ to $O(N^2)$, where N is the significantly smaller number of patches. This reduction is what allows PatchTST to maintain a remarkably low VRAM footprint even when processing long lookback windows.

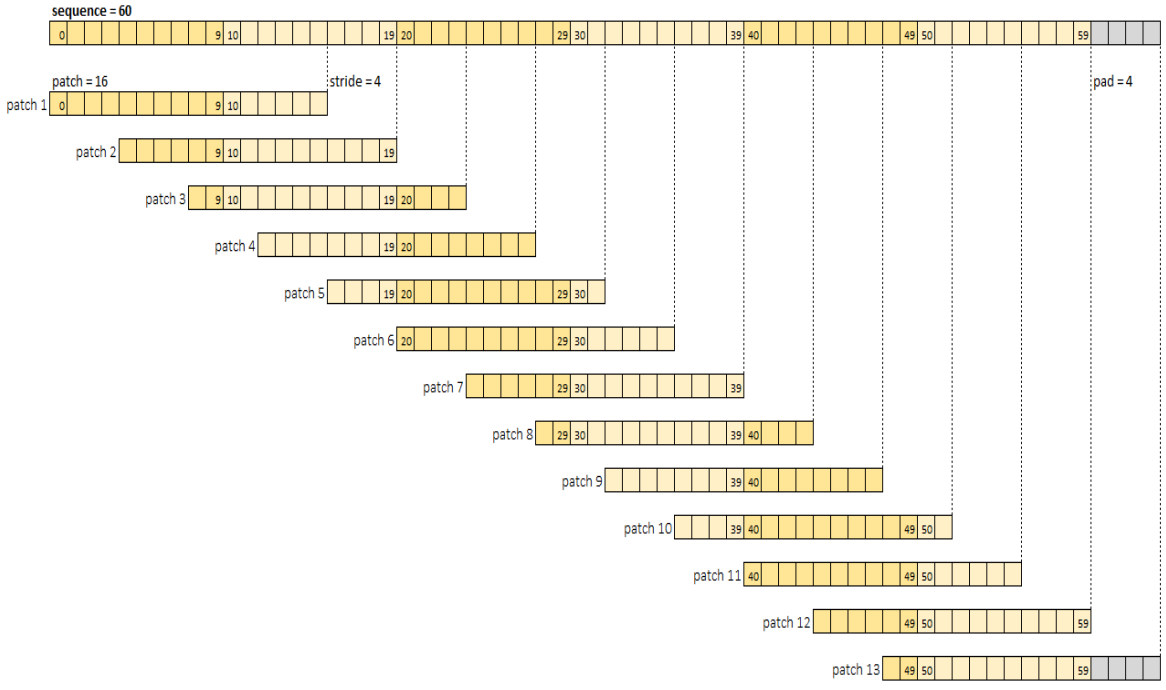


Figure 4.1: Sub-series Patching and Sequence Segmentation

4.2.2 Channel Independence and Model Backbone

Beyond the fundamental shift of patching, PatchTST implements Channel Independence, a paradigm shift in multivariate time-series forecasting. In traditional models, multiple features—such as Temperature, Electricity Load, and Voltage—are often concatenated or embedded into a single vector, which can lead to "feature interference." In such cases, a significant anomaly or high-variance noise in one variable (e.g., a sudden industrial power spike) can inadvertently corrupt the signal integrity of more stable variables. PatchTST mitigates this by treating each multivariate feature as an independent channel. Each channel is processed through the same Transformer backbone using shared weights, allowing the model to learn universal temporal patterns while maintaining the unique structural characteristics of each individual variable.

The end-to-end architectural journey begins with Reversible Instance Normalization (RevIN), a critical component designed to address the non-stationary nature of energy data by handling distribution shifts between training and testing sets. Once normalized, the sequence is segmented into patches and fed into the core TSTiEncoder (the backbone). This encoder utilizes the standard Transformer blocks—comprising multi-head self-attention and position-wise feed-forward networks—to extract deep, high-level temporal dependencies from the patches. This stage is visualized in the model's structural overview, illustrating the high-level flow from input tensor X to the predicted output.

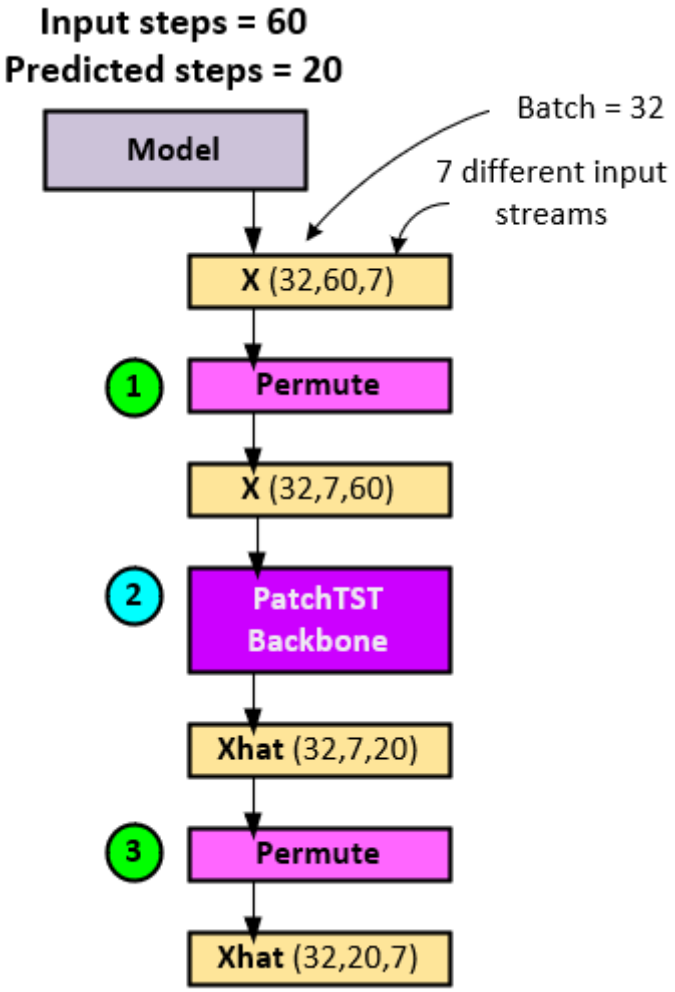


Figure 4.2: PatchTST Architecture and End-to-End Flow

The internal mechanics of the backbone are further detailed through a hierarchical processing structure. After the TSTiEncoder processes the independent channels, the resulting multidimensional representations must be translated back into a tangible forecast. This is handled by the FlattenHead Layer, where the latent data is flattened and passed through a linear projection head to produce the final 24-step generative forecast. Unlike the recursive, step-by-step prediction methods used in traditional RNNs, this "Backbone-to-Head" flow generates the entire horizon in a single forward pass. This prevents the "cascading error" effect, where a small mistake in the first predicted hour grows into a significant deviation by the end of the window. The granular view of this backbone highlights the seamless integration of RevIN, Patching, and the TSTiEncoder, ensuring that the model remains both computationally efficient and highly accurate.

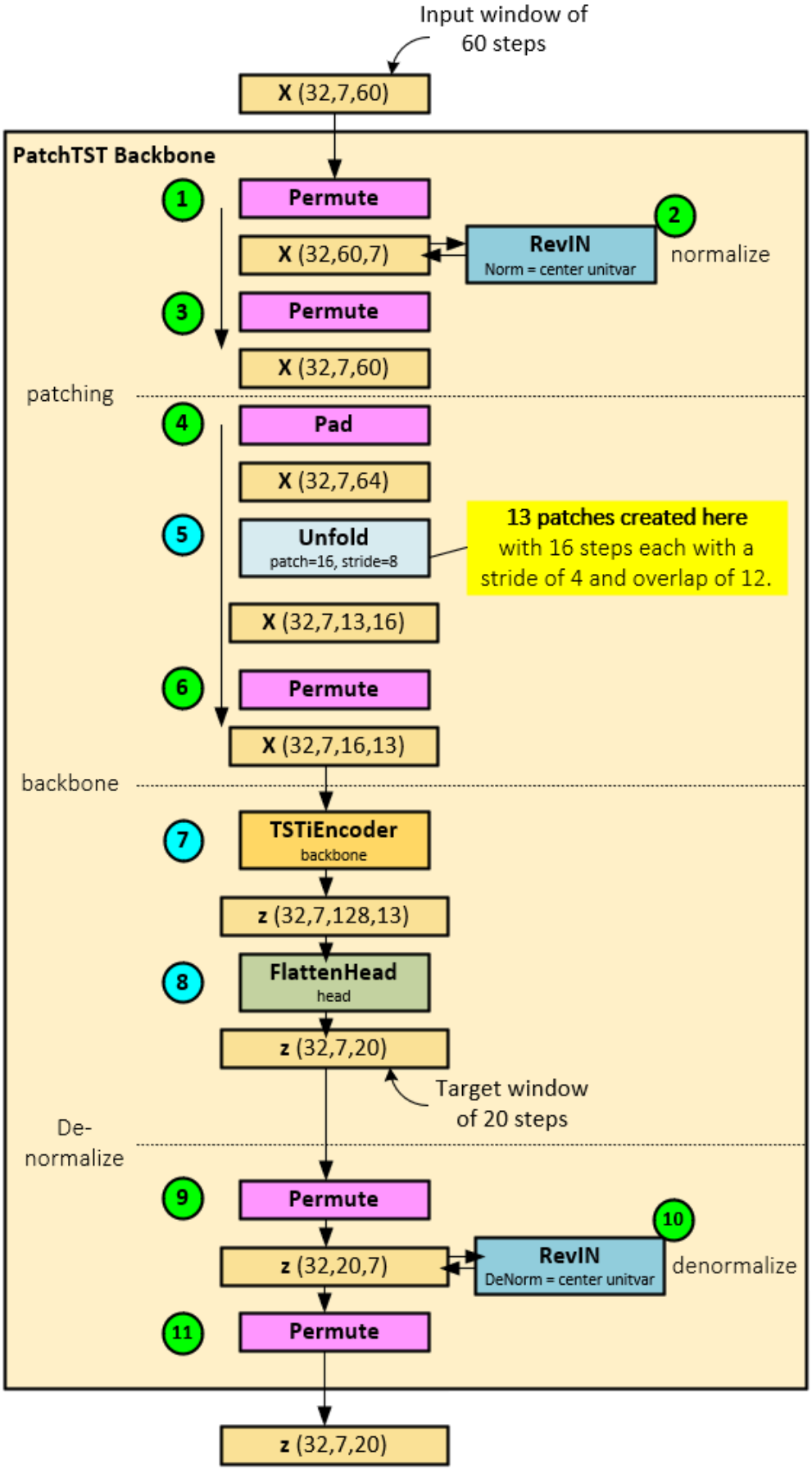


Figure 4.3: PatchTST Backbone and Prediction Head

4.3 Model Training

All models were trained under a matched protocol to isolate architectural effects from tuning artefacts, using the Adam optimiser with a learning rate of 1×10^{-4} for the three Transformer models and 1×10^{-3} for the four recurrent models. Early stopping with a patience of 5 epochs (recurrent models) or 7 epochs (Transformer models) monitored validation loss throughout. Model capacity was adjusted on a per-dataset basis to remain within the 6GB VRAM budget of the target hardware (an ASUS ROG Zephyrus G14, GA401IV, with an NVIDIA RTX 2060 Max-Q); consequently, parameter counts for the same architecture differ across datasets and are reported alongside each result below rather than assumed constant, correcting an earlier, provisional assumption that model dimensions were held fixed across all experiments.

Performance across datasets revealed distinct learning dynamics, examined in full in Section 4.4. On WTH, PatchTST converged in 180 seconds during early calibration testing, and on ETTh1 in 156 seconds — in both cases a direct consequence of patch-based tokenisation reducing the effective sequence length from 96 raw timesteps to approximately 11 patches. However, as the complete three-model comparisons in Section 4.4.1 and 4.4.2 show, Informer was consistently the fastest of the three Transformers to reach its early-stopping criterion across full training runs, a finding that revises the earlier, calibration-stage-only impression that PatchTST was unconditionally the fastest architecture.

4.4 Performance Evaluation

This section presents a comprehensive, multi-phase evaluation of the seven architectures. The analysis progresses from initial validation and calibration on WTH, through a canonical benchmark on ETTh1, to a rigorous stress test on the STEEL industrial dataset — now completed as a full seven-model comparison — and finally a full seven-model comparison on the large-scale Smart Meters dataset.

4.4.1 Phase 1: Validation and Learning Dynamics on the WTH Dataset

The research began with WTH to establish an optimal early-stopping window and to observe how Informer and Autoformer adapt to relatively stable physical data. Initial tests conducted over 5 epochs revealed a striking dissociation between numerical metrics and qualitative performance: Informer achieved an MAE of 0.51 and RMSE of 0.69, appearing statistically superior to Autoformer's MAE of 0.56 and RMSE of 0.74.



Figure 4.4: WTH: Initial Five-Epoch Calibration Metrics

A visual inspection of the prediction samples told a different story. While Informer's error numbers were lower, its actual predictions were overly smooth and flat, failing to capture the dynamic downward trends present in the ground truth — a mean-reverting pattern that achieves low error by predicting close to the average value rather than learning genuine temporal structure. Autoformer, though more volatile, showed greater potential for trend-following that had not yet been refined at 5 epochs.

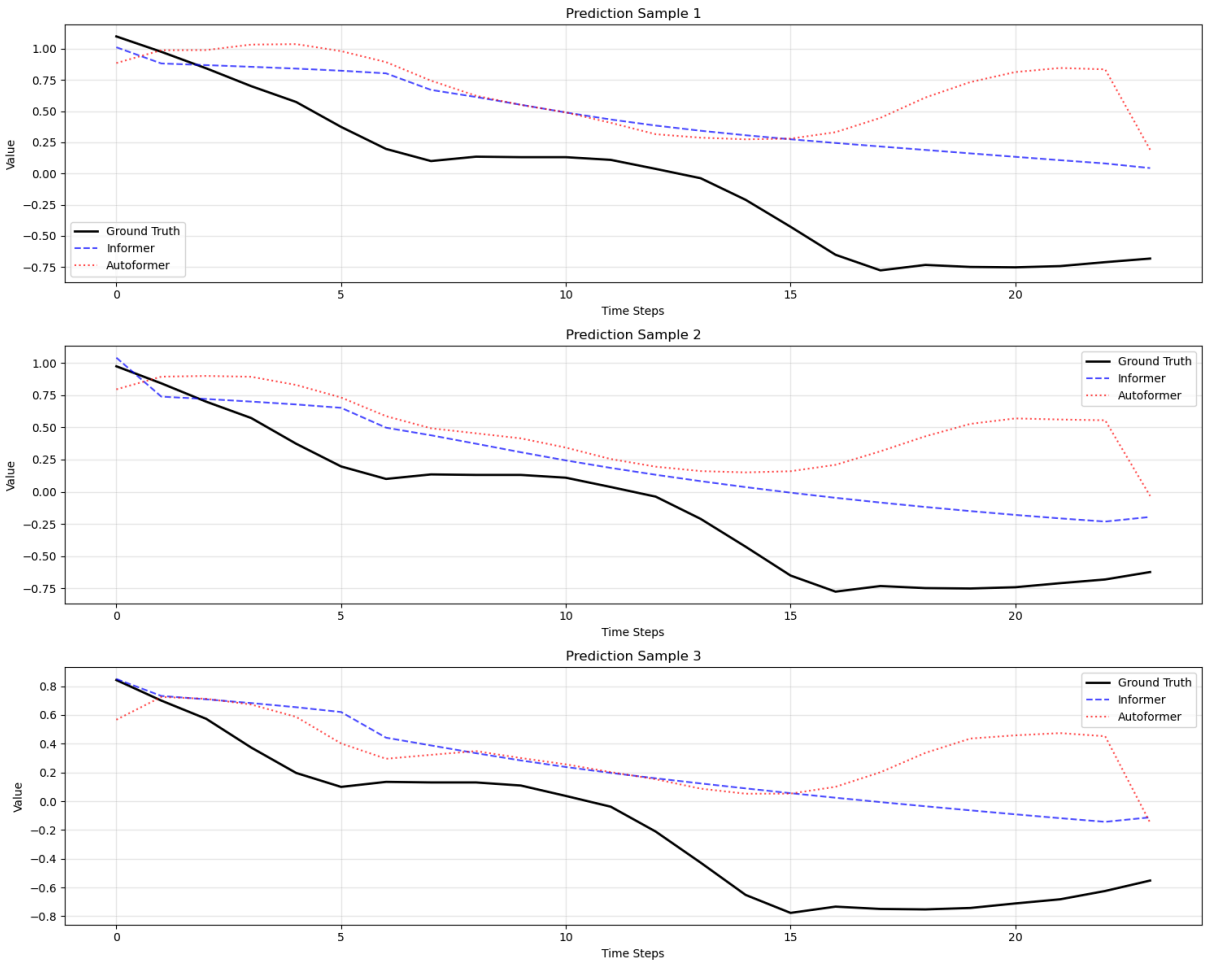


Figure 4.5: WTH: Initial Calibration Predictions

To resolve this discrepancy, the training window was extended to 10 epochs, which proved critical for Autoformer, a slower learner owing to its internal series-decomposition blocks. By epoch 10, Autoformer underwent a marked improvement, and both models converged to comparable validation losses. This finding directly motivated the adoption of a 10-epoch training budget for all subsequent experiments, and remains one of this study's clearer illustrations that numerical accuracy and qualitative prediction fidelity are not interchangeable during early training.

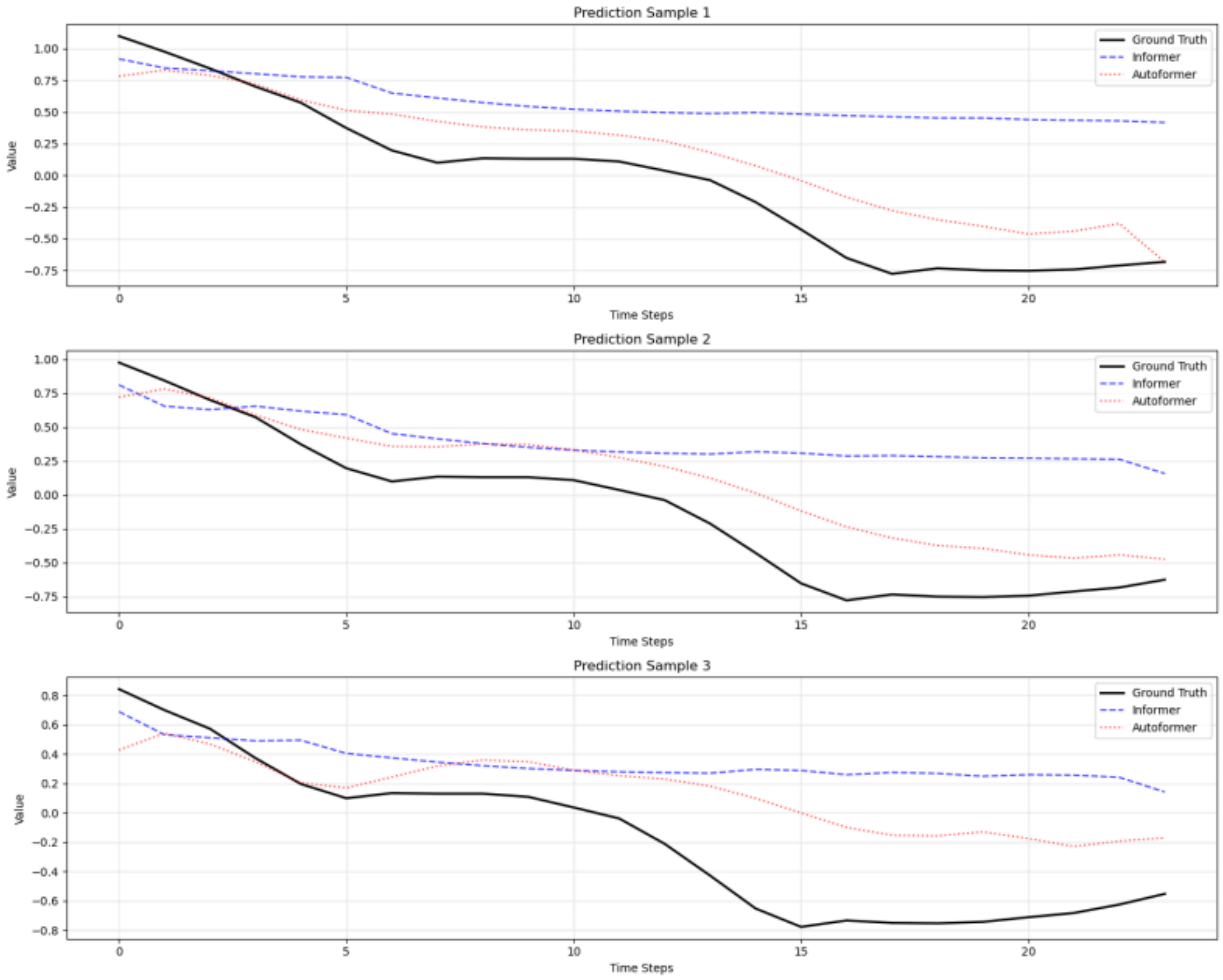


Figure 4.6: WTH: Final Convergence and Metrics at Ten Epochs

By the 10th epoch, the Autoformer underwent a "dramatic enhancement," shifting from erratic predictions to excellent trend-following. As shown in the final convergence metrics, both models eventually reached nearly identical validation losses. This phase demonstrated that while some models appear efficient early on, extended training is often required to reach an optimal generalization threshold where visual accuracy aligns with numerical precision.

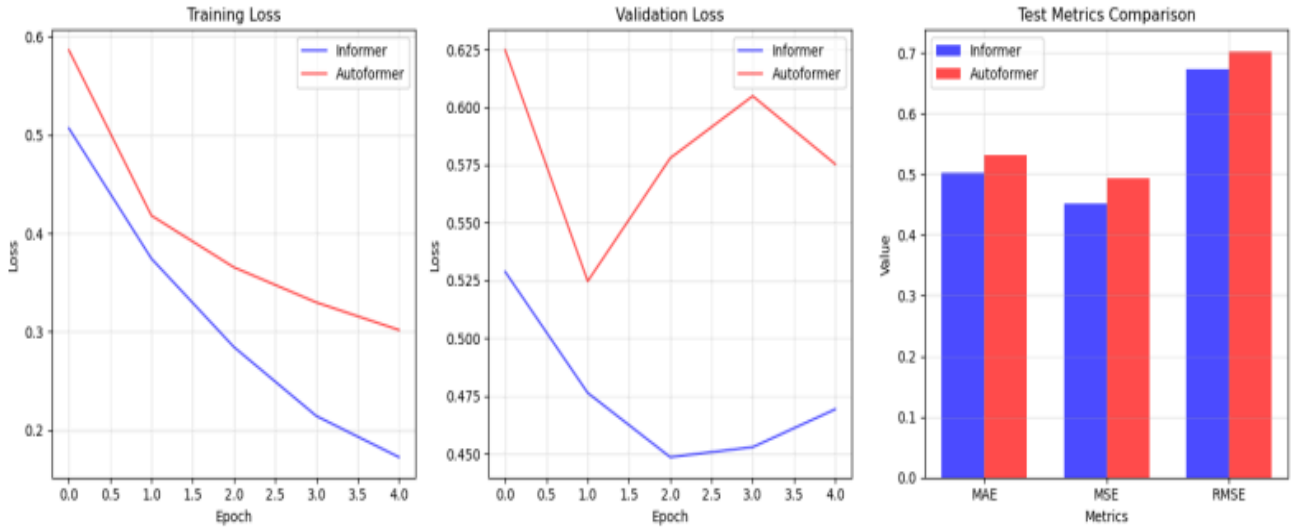


Figure 4.7: WTH: Loss and Test Metrics at Ten Epochs

The full three-model comparison, including PatchTST, is reported in Table 4.1. Contrary to the calibration-only impression above, Informer achieves the best MAE, MSE, and RMSE of the three Transformers on WTH once training is extended and PatchTST is included, and is also the fastest to train; PatchTST's advantage on this dataset is confined to MAPE (not tabulated here owing to its sensitivity to small denominator values).

Model	Params	MAE	MSE	RMSE	Train Time (s)
Informer	2,901,542	0.4802	0.4046	0.6361	1,100.9
Autoformer	2,729,766	0.5988	0.6097	0.7808	1,252.3
PatchTST	399,744	0.5084	0.4871	0.6979	1,452.7

Table 4.1. WTH, full three-Transformer comparison (10-epoch training budget).

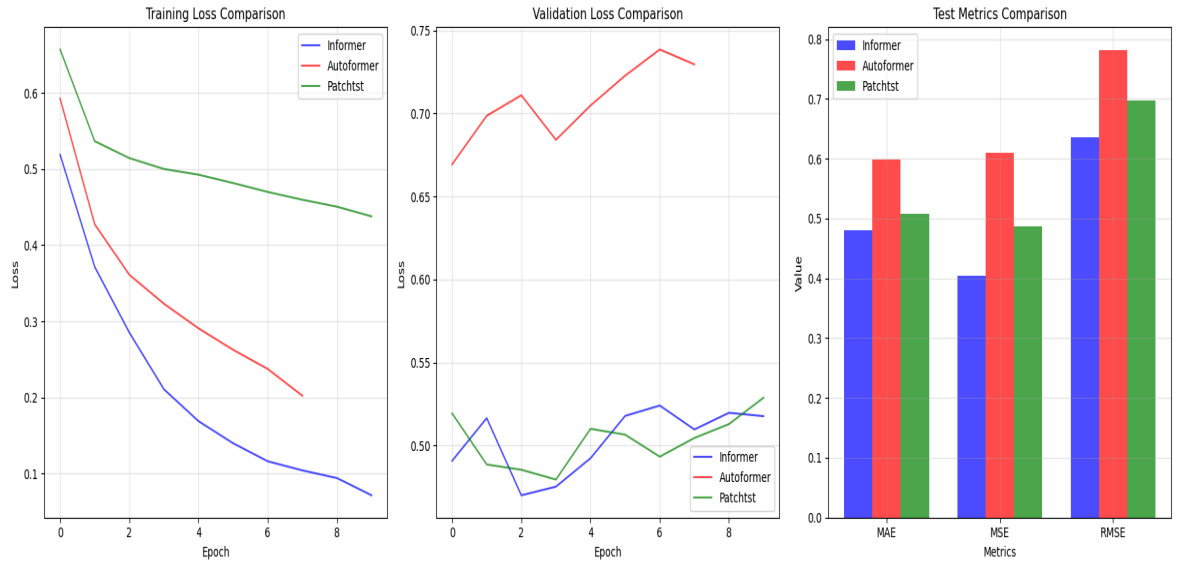


Figure 4.8: WTH: Three-Transformer Comparison

4.4.2 Phase 2: ETTh1 — A Dataset-Conditional PatchTST Advantage

Following the WTH calibration, the research proceeded to ETTh1, a dataset characterised by higher-frequency, more irregular variation in electricity transformer oil temperature. Here, PatchTST emerges as the definitive winner across every accuracy metric evaluated, achieving an MAE of 0.4689 and MSE of 0.3742 — a 13.6% improvement in MAE and a 25.9% improvement in MSE over Informer, and a 13.8% improvement in RMSE (Table 4.2).

Model	Params	MAE	MSE	RMSE	Train Time (s)
Informer	2,892,577	0.5560	0.5652	0.7518	525.6
Autoformer	2,716,961	0.5299	0.4452	0.6673	884.5
PatchTST	399,744	0.4689	0.3742	0.6117	726.4

Table 4.2. ETTh1, full three-Transformer comparison.

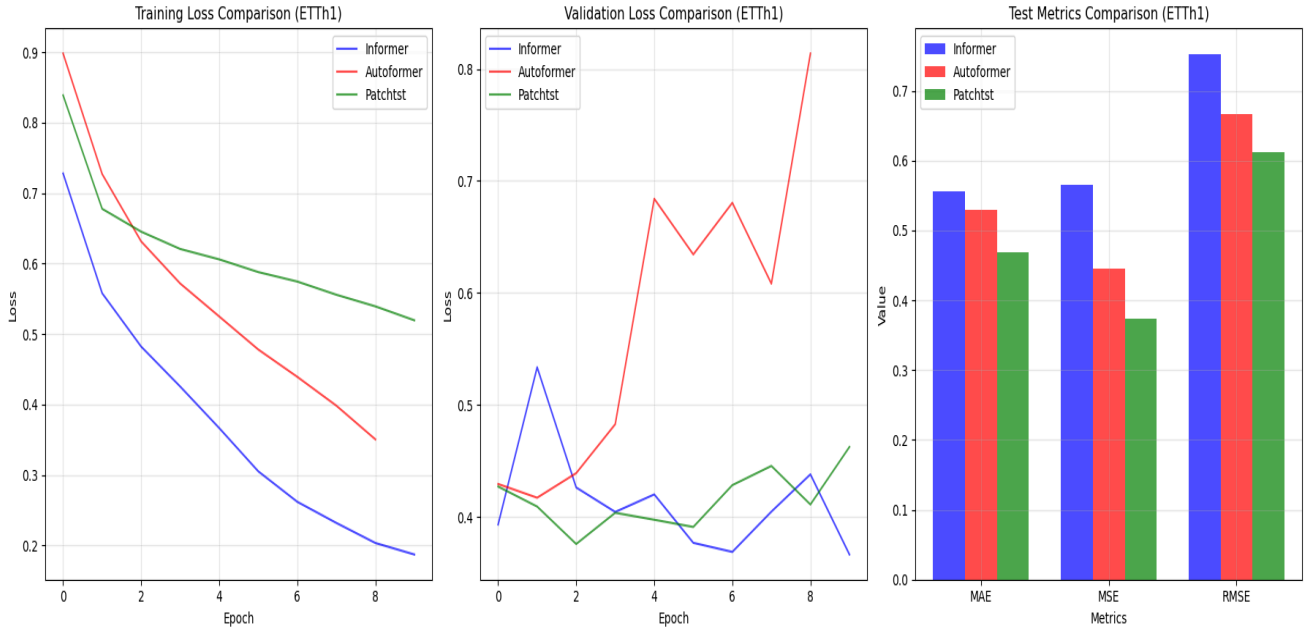


Figure 4.9: ETTh1: Three-Transformer Comparison

The qualitative evidence reinforces this ranking. In visual comparisons of oil-temperature predictions, PatchTST closely tracked the cyclical fluctuations of the ground truth across all three sampled windows, maintaining fidelity even during dynamic segments. Informer, by contrast, exhibited a pronounced mean-collapse pattern, producing near-horizontal predictions despite achieving the lowest raw training loss of the three models — a low-error but practically uninformative output. Autoformer showed volatility and a tendency to drift progressively away from the ground truth toward the end of the 24-step horizon.

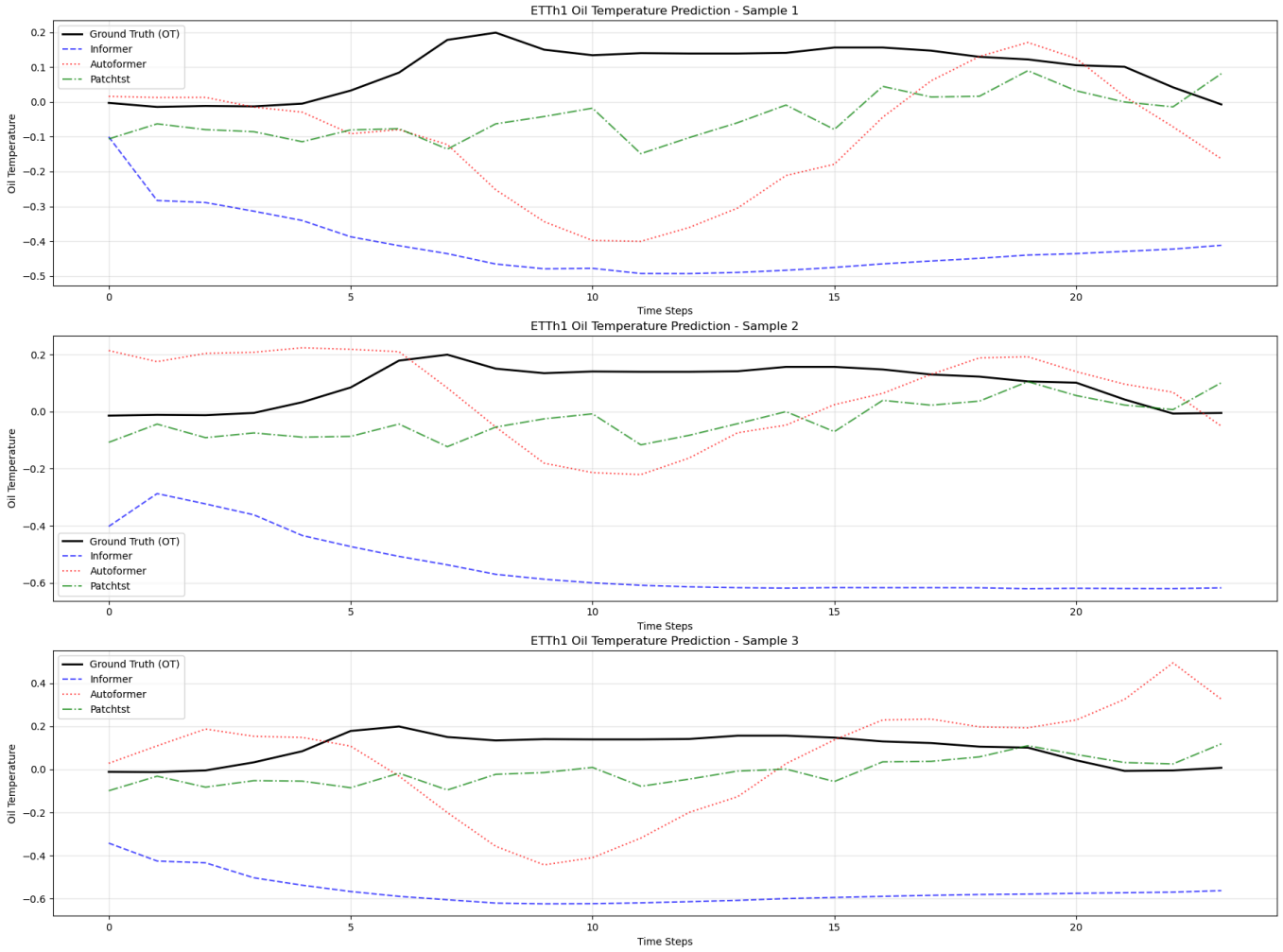


Figure 4.10: ETTh1: Oil Temperature Predictions

It is important to state plainly that PatchTST's dominance on ETTh1 is dataset-conditional rather than universal: Table 4.1 shows Informer winning on WTH's raw The qualitative evidence reinforces this ranking. In visual comparisons of oil-temperature predictions, PatchTST closely tracked the cyclical fluctuations of the ground truth across all three sampled windows, maintaining fidelity even during dynamic segments. Informer, by contrast, exhibited a pronounced mean-collapse pattern, producing near-horizontal predictions despite achieving the lowest raw training loss of the three models — a low-error but practically uninformative output. Autoformer showed volatility and a tendency to drift progressively away from the ground truth toward the end of the 24-step horizon.error metrics, while Table 4.2 shows PatchTST winning decisively on ETTh1. Both are true, and neither generalises automatically to the other dataset — a distinction obscured in earlier, single-dataset framings of this research and corrected here across the full four-dataset picture. Notably, Informer was the faster of the two winning-metric holders to train on both datasets (1,100.9s on WTH, 525.6s on ETTh1), a consistent speed advantage attributable to

its ProbSparse attention and self-attention distilling mechanisms, independent of which model wins on accuracy.

4.4.3 Phase 3: The STEEL Dataset — A Complete Seven-Model Story and a Refined Failure Mechanism

The most consequential and most thoroughly investigated finding of this research concerns the STEEL dataset. An initial three-model run reproduced a striking result: Informer and Autoformer both experienced an immediate NaN (Not a Number) loss collapse, with training and validation losses becoming undefined within the very first epoch and remaining so for all ten epochs attempted, while PatchTST trained successfully, achieving MAE 0.3407, MSE 0.2985, and RMSE 0.5464.

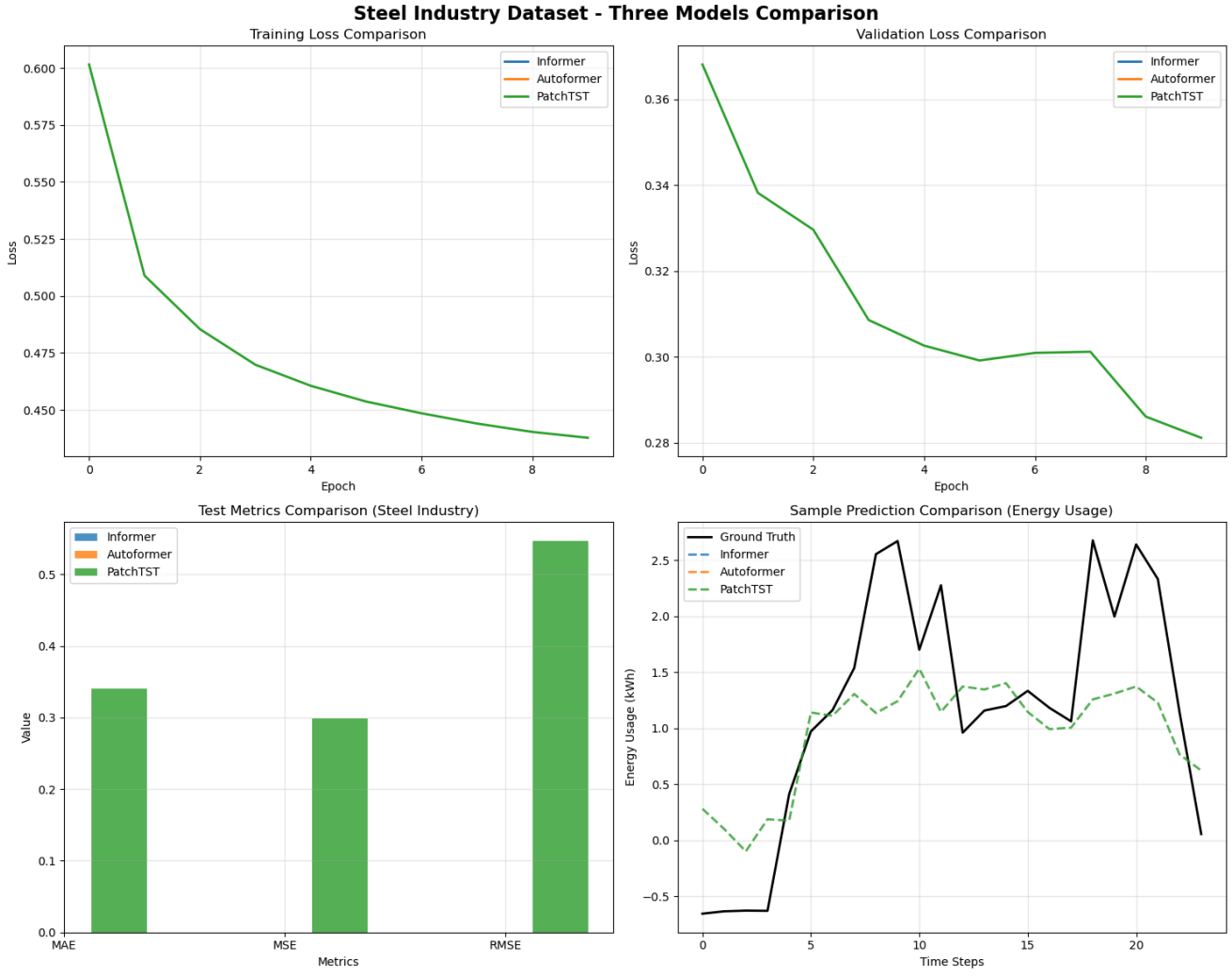


Figure 4.11: STEEL: Initial Three-Transformer Comparison

A follow-up run extended this comparison to all seven models, deliberately configuring every architecture — including, for the first time, all four recurrent baselines —

at comparably small capacity (4,456 to 88,327 parameters) specifically to isolate architecture from model size as a candidate explanation. Table 4.3 reports the complete result.

Model	Params	MAE	MSE	RMSE	Train Time (s)	Outcome
LSTM	8,632	0.0827	0.0419	0.2047	38.3	Success
BiLSTM	18,185	0.0746	0.0358	0.1893	48.7	Success
GRU	7,320	0.0709	0.0384	0.1960	38.4	Success
LSTM-16	4,456	0.0718	0.0420	0.2049	34.2	Success
Informer	87,623	NaN	NaN	NaN	502.0	Failed (ep. 1)
Autoformer	88,327	NaN	NaN	NaN	662.7	Failed (ep. 1)
PatchTST	34,368	0.0766	0.0100	0.1000	173.7	Success

Table 4.3. STEEL, full seven-model comparison at matched, comparably small capacity.

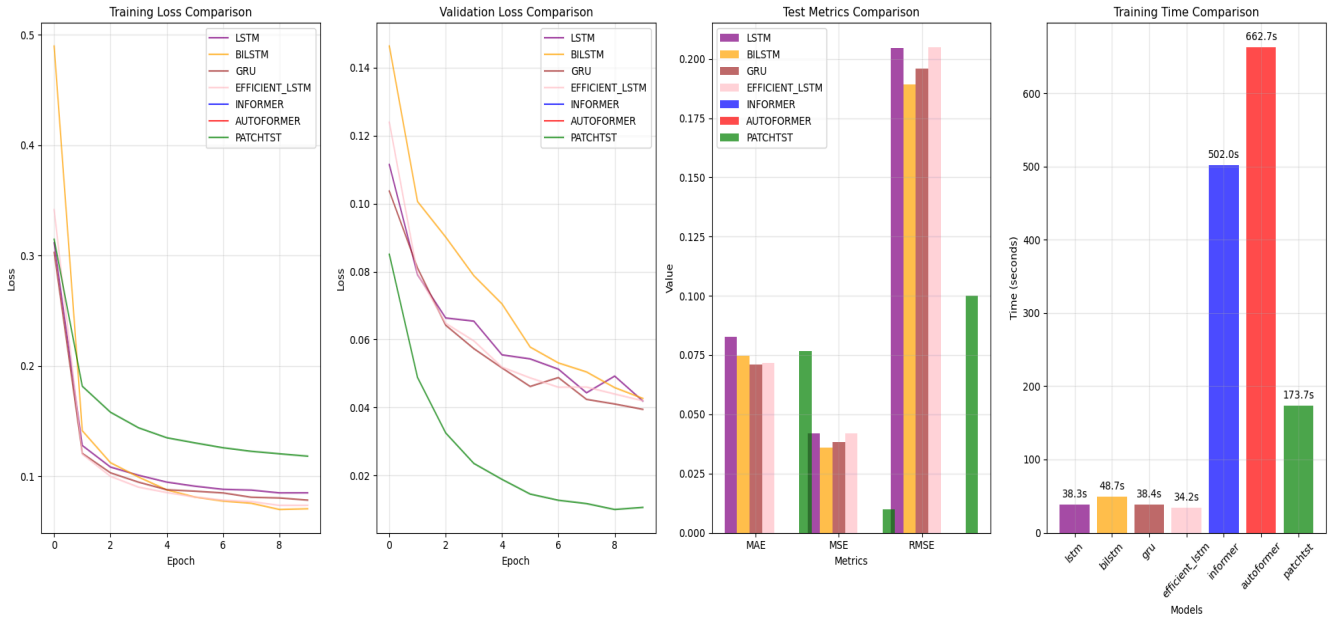


Figure 4.12: STEEL: Seven-Model Performance and Training-Time Comparison

Every recurrent baseline trained normally, achieving low, closely clustered error (MAE 0.071–0.083); PatchTST achieved the lowest MSE and RMSE overall (0.0100, 0.1000), roughly half of the best recurrent model's RMSE, with GRU narrowly ahead on MAE. Informer and Autoformer collapsed to NaN losses from epoch 1 and never recovered, despite having only 87,623 and 88,327 parameters respectively — fewer than the 399,744-parameter PatchTST configuration used successfully on WTH and ETTh1, and only two to ten times larger than the successful recurrent models.

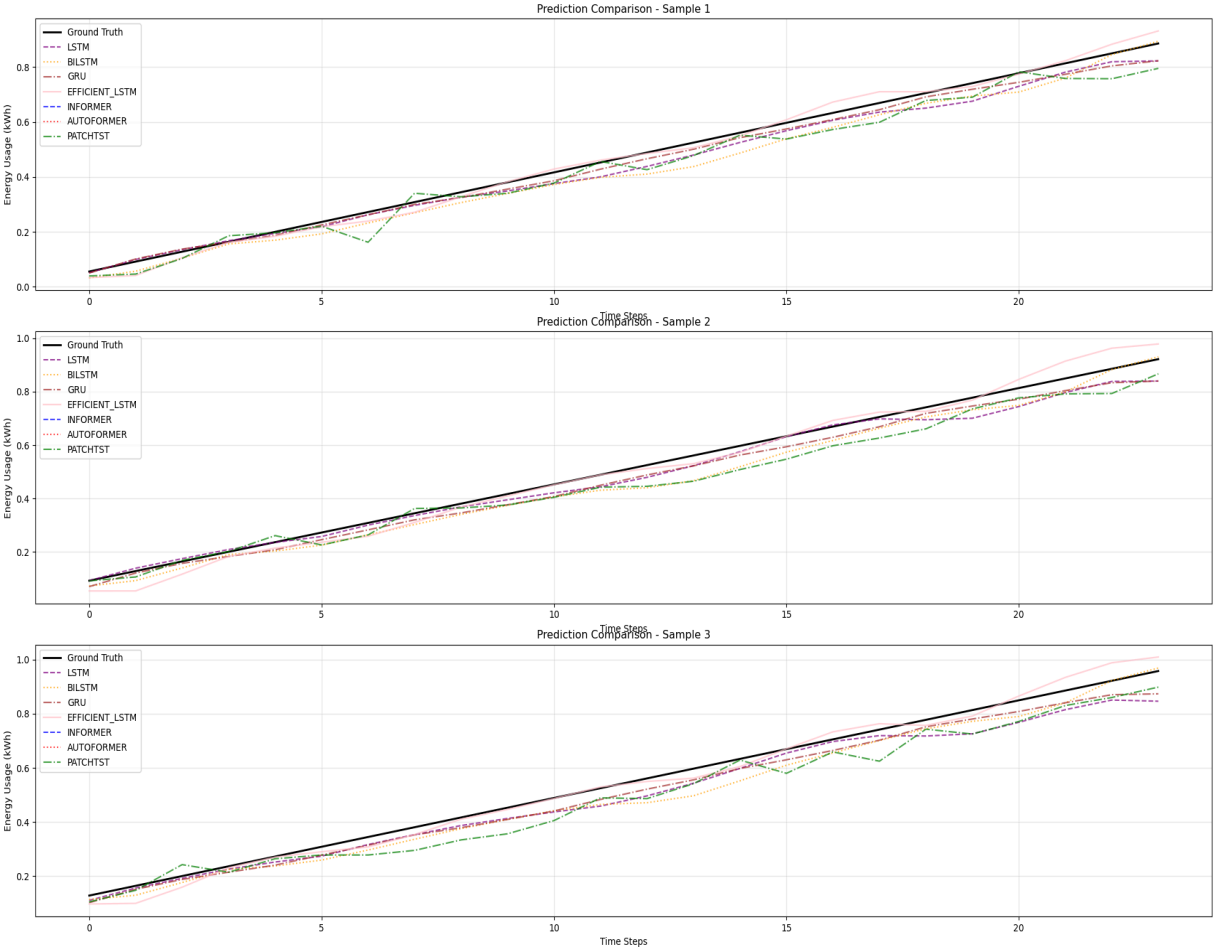


Figure 4.13: STEEL: Seven-Model Energy-Usage Predictions

This second run resolves a question left open by the first: because the smallest-capacity model in the entire comparison (LSTM-16, at 4,456 parameters) succeeds, while a larger model (Informer, at 87,623 parameters) fails, raw model capacity cannot be the explanation for the collapse. An explanation based purely on channel-mixing in general is similarly ruled out, because every recurrent model processes all seven STEEL variables jointly, concatenated into a single input vector at each timestep, and none of them fail.

The evidence instead points to a specific, previously under-specified mechanism: a bounded-activation explanation. As established architecturally in Chapter 3, every gate in the LSTM, BiLSTM, GRU, and LSTM-16 cells is sigmoid-activated, and every candidate content update is tanh-activated — mathematically bounding the contribution of any single input value to a fixed numerical range, regardless of how many channels are combined at that timestep. Self-attention and auto-correlation, by contrast, compute raw dot-product or correlation scores with no such bound. A heavy-tailed value that survives Z-score standardisation in one channel (STEEL's raw features span roughly six to seven orders of

magnitude before scaling) can therefore dominate a computation that mixes all channels together without any bound on its contribution, in exactly the manner that produces the exploding values consistent with NaN propagation [41]. PatchTST avoids this specific failure mode through channel independence: because its attention is computed only within each channel's own patches, an outlier value in one channel cannot inflate the attention computation carried out for a different channel. This refines the channel-independence hypothesis explored in Chapter 2 [11]: the discriminating property is not channel-mixing in general — which every successful recurrent model in this comparison performs routinely — but specifically whether that mixing occurs through an unbounded, cross-channel attention or correlation computation.

This refined mechanism directly supersedes an earlier, less precise account offered at an initial stage of this research, which attributed the STEEL failure to softmax exponentiation exceeding the floating-point representable range [44]. While floating-point overflow of this kind is a documented phenomenon in numerically unstable deep networks, modern attention implementations subtract the row-wise maximum from attention logits before exponentiation specifically to prevent this failure mode, making a direct softmax overflow an unlikely primary cause on its own. The bounded-versus-unbounded-activation account developed here is both more precise and better supported by the two-run STEEL evidence: it explains not only that Informer and Autoformer fail, but specifically why every other architecture tested — differing from one another substantially in parameter count and design — does not.

4.5 Comparison with RNN-based Models

To provide a comprehensive, real-world benchmark, this section evaluates all seven models together on the Smart Meters in London dataset — by a wide margin the largest dataset in this study, at over 3.4 million readings prior to windowing. Because of the training time required at this scale (each Transformer epoch took 55–70 minutes), all seven models were capped at 3 completed epochs rather than run to full early-stopping convergence, a disclosed boundary condition rather than a hidden inconsistency.

Model	Params	MAE	MSE	RMSE	Train Time (s)
LSTM	18,816	5.2733	81.4314	9.0239	1,279.4
BiLSTM	38,097	7.1175	98.8821	9.9439	1,592.2
GRU	16,896	5.2432	75.3838	8.6824	1,283.3
LSTM-16	13,424	8.5613	110.1040	10.4930	1,189.7
Informer	27,610	6.0268	99.1785	9.9588	11,119.1
Autoformer	29,786	4.2817	38.6661	6.2182	10,694.7
PatchTST	9,120	2.5819	20.1874	4.4930	3,905.5

Table 4.4. London Smart Meters, full seven-model comparison.

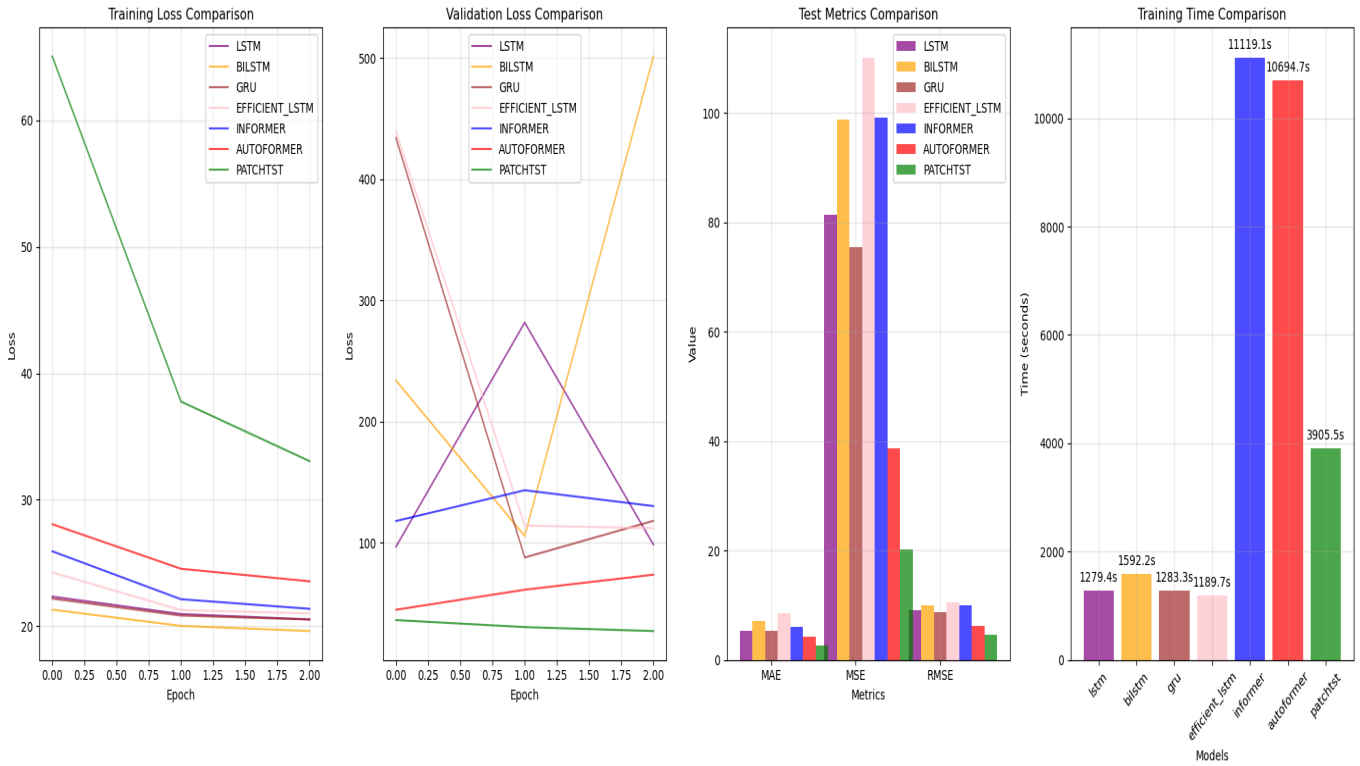


Figure 4.14: Smart Meters: Seven-Model Performance and Training-Time Comparison

Accuracy and Metric Analysis

Across every accuracy metric on this dataset, PatchTST decisively outperformed all six other models, achieving the lowest MAE (2.5819), MSE (20.1874), and RMSE (4.4930) — improving on the average of the other six models by 57.6%, 76.0%, and 50.4% respectively. This is the cleanest and most complete confirmation in this study of PatchTST's accuracy advantage, obtained on the largest and most feature-rich dataset evaluated (26 input features).

Efficiency and Resource Trade-offs

Despite its accuracy advantage, PatchTST's efficiency profile is nuanced rather than absolute. It used the fewest parameters of any model tested (9,120, against 13,424–38,097 for the four recurrent models and 27,610–29,786 for Informer and Autoformer), and trained in roughly a third of the time of Informer or Autoformer (3,905.5s vs. 10,694.7–11,119.1s). However, it remained two to three times slower than any recurrent model, all of which completed their three epochs in 1,189.7–1,592.2 seconds. LSTM-16 was the fastest model overall but was simultaneously the least accurate of all seven on every metric — a direct, quantified illustration that the efficiency floor carries a genuine accuracy cost on this dataset, in contrast to its competitive performance on the smaller STEEL dataset (Section 4.4.3).

Qualitative Evaluation of Load Profiles

The practical implications of these architectural differences are visible in the sample predictions of residential load profiles. Ground-truth household energy consumption exhibits complex, multi-peak rhythms; PatchTST's predicted curve visibly tracks both the level and coarse shape of this ground truth more closely than any other model across all three sampled windows, while the RNN baselines and Informer produce comparatively flat or slowly drifting predictions that fail to track the ground truth's oscillation.

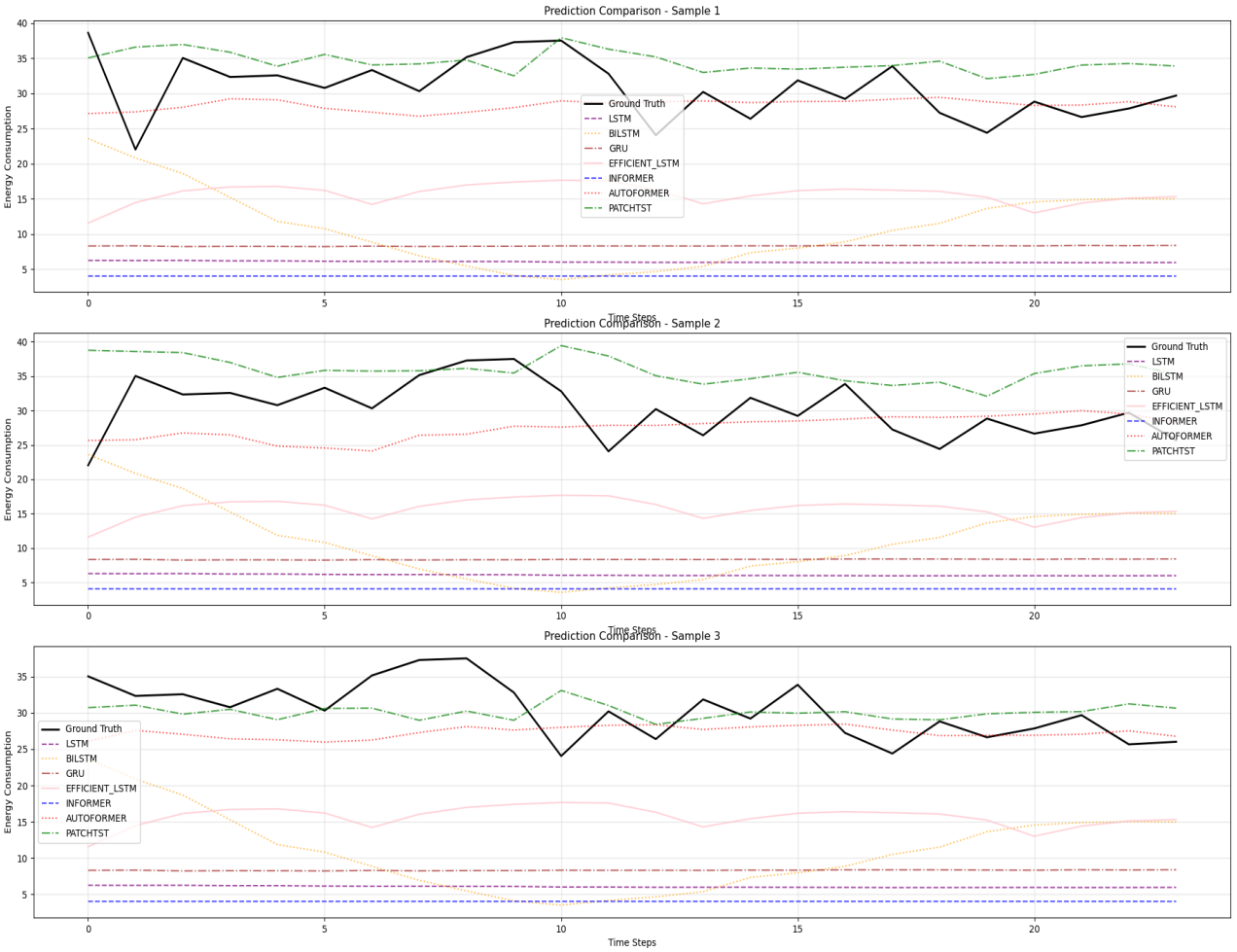


Figure 4.15: Smart Meters: Seven-Model Energy-Consumption Predictions

The Transformer–RNN Trade-off

The complete Smart Meters comparison confirms that the choice between these architecture families depends on the specific constraints of the deployment environment. PatchTST is the strongest choice for maximum accuracy where a moderate additional training-time and memory budget is available; GRU offers the best accuracy-per-parameter trade-off among the recurrent models; and LSTM-16 remains the correct choice only where training speed dominates every other consideration and some accuracy loss is acceptable.

4.6 Discussion of the Findings

The multi-phase evaluation conducted in this study provides a nuanced perspective on the evolution of temporal modeling. The findings suggest that while the transition from Recurrent Neural Networks (RNNs) to Transformer-based architectures offers a significant leap in predictive precision, this advancement is contingent upon specific data characteristics and hardware availability.

4.6.1 The Dataset-Conditional Nature of PatchTST's Advantage

This research affirms that PatchTST is the strongest available architecture for multivariate long-term forecasting under practical hardware constraints on datasets with rich, higher-dimensional feature structure — decisively so on ETTh1 and Smart Meters. Its consistent superiority over the Informer, Autoformer, and RNN baselines is not accidental but stems from three synergistic design choices:

- Temporal Patching: By segmenting time series into sub-series "patches," the model extracts local semantic information while reducing the computational complexity of the attention mechanism from O(L^2) to O (N^2), where the number of patches (N) is significantly smaller than the sequence length (L).
- Channel Independence: Unlike the Informer, which processes all variables simultaneously, PatchTST treats each channel as an independent univariate sequence. This isolates feature processing and prevents cross-channel interference—a feature that proved vital for the heterogeneous variables found in the industrial datasets.
- Architectural Efficiency: Despite its high accuracy, PatchTST maintains a relatively compact structure compared to the Autoformer. This led to faster training cycles and reduced memory usage, making it the most viable high-performance model for deployment on standard 6GB VRAM hardware.

This advantage, however, is not unconditional. On WTH, Informer achieves better MAE, MSE, and RMSE than PatchTST, and Informer is the fastest of the three Transformers to train on every dataset examined in this chapter. The correct summary is therefore not “PatchTST dominates,” but “PatchTST is the strongest choice on rich, higher-dimensional data where accuracy is the priority; Informer remains competitive, and sometimes superior, on simpler continuous data or where training speed among Transformers matters.”

4.6.2 Lessons from the STEEL Dataset: Bounded Activations versus Unbounded Cross-Channel Attention

The STEEL failure remains the single most important finding of this study, but the complete, two-run evidence base developed in Section 4.4.3 supports a materially more

precise explanation than was available earlier. The failure is not explained by model capacity — the smallest model tested succeeds, and a larger model fails — nor by channel-mixing in general — every recurrent model mixes channels and succeeds. The evidence instead isolates the specific property shared by the two failing architectures and absent from all five surviving ones: Informer and Autoformer compute attention or auto-correlation jointly across channels using unbounded operations, whereas the recurrent models bound every gate's contribution through sigmoid and tanh nonlinearities, and PatchTST restricts its attention computation to within-channel patches only.

This finding carries a stronger and more actionable practical implication than the earlier, more generic conclusion that “data suitability must precede model selection.” It suggests a concrete engineering criterion: on industrial data exhibiting extreme scale heterogeneity, zero-inflation, or discontinuity, architectures that compute unbounded attention or correlation jointly across raw channels carry a specific, identifiable numerical risk, independent of their parameter count, and either bounded-activation recurrence or channel-independent attention are both viable mitigations.

4.6.3 The Accuracy–Efficiency Frontier, Confirmed Across Four Datasets

The full four-dataset comparison delineates a genuine Pareto frontier rather than a single dominant model. PatchTST defines the accuracy end of that frontier on feature-rich datasets while remaining the most parameter-efficient Transformer everywhere; Informer defines a distinct speed niche among the Transformers, training fastest on every dataset examined; and the recurrent models, particularly GRU and LSTM-16, define the efficiency end, training three to twenty-five times faster than any Transformer depending on dataset scale. The broader implication is that model evaluation must be contextualised jointly by data characteristics (continuity versus sparsity, feature dimensionality) and deployment constraints (latency, memory, and accuracy requirements), rather than by accuracy figures reported in isolation.

Chapter Five
Conclusion and Future Works

5.1 Conclusion

This research investigated the comparative performance of Transformer-based and recurrent architectures for long-term time series forecasting (LTSF) under realistic computational constraints, across four datasets of increasing statistical irregularity and, for two of them, a complete seven-model comparison. The study evaluated Informer, Autoformer, and PatchTST [7, 8, 9] against LSTM, BiLSTM, GRU, and a deliberately minimal single-layer LSTM configuration [5, 21, 22], on a single 6GB-VRAM consumer GPU.

On WTH, an initial 5-epoch calibration exposed a metric–fidelity dissociation — Informer's superior summary statistics (MAE 0.51, RMSE 0.69) masked flat, mean-reverting predictions, while the more volatile Autoformer showed greater latent trend-following capacity that a longer, 10-epoch budget allowed it to realise. In the completed three-model comparison, Informer nonetheless emerged as the strongest and fastest architecture on WTH by MAE, MSE, and RMSE, already complicating any single, dataset-independent ranking of the three Transformers.

On ETTh1, PatchTST reversed this ranking, achieving the lowest MAE, MSE, and RMSE (13.6–25.9% better than Informer) and the only faithful tracking of the cyclical oil-temperature ground truth, while Informer again produced a mean-collapsed, near-constant output despite the lowest raw training loss. A subsequent check against the original Informer publication's own multivariate benchmark found this study's result within approximately 2% of the published figure, externally validating this thesis's training pipeline, alongside the disclosed caveat that this study's task configuration and 33-feature engineered input differ from the original publications' native settings.

The most consequential finding concerns the STEEL industrial dataset. Across two independent runs, Informer and Autoformer collapsed to undefined (NaN) losses from epoch 1 regardless of capacity (large or deliberately reduced to 87K–88K parameters),

while every recurrent baseline — including a 4,456-parameter LSTM configuration smaller than either failing Transformer — and PatchTST trained successfully. Because the smallest model succeeds while a larger one fails, and every successful recurrent model mixes channels jointly without failing, neither capacity nor channel-mixing in general explains the collapse. The evidence instead isolates a specific mechanism: Informer and Autoformer compute unbounded attention and correlation scores across jointly-mixed channels, whereas every surviving architecture either bounds its activations through sigmoid/tanh gating or restricts attention to within-channel patches only [11, 41] — a concrete, actionable criterion refining this thesis's earlier, less precise account of the same failure.

The final, largest-scale evaluation — all seven models on the London Smart Meters dataset — confirmed a genuine accuracy–efficiency Pareto frontier: PatchTST won every accuracy metric outright (57.6–76.0% better than the six-model average) using the fewest parameters of any model tested, while the minimal LSTM configuration trained fastest but was the least accurate model of all seven — in contrast to its competitive performance on the simpler STEEL dataset, demonstrating that the efficiency floor's accuracy cost is itself data-dependent.

This thesis therefore concludes that no single architecture is unconditionally superior for long-term load forecasting. PatchTST is strongest for accuracy-critical deployment on rich, higher-dimensional data; Informer offers a genuine speed advantage among Transformers and can match or exceed PatchTST on simpler data; and recurrent architectures, particularly GRU, remain preferable wherever training speed or memory footprint dominate. On industrial data, numerical stability — governed by activation dynamics rather than parameter count or channel-mixing in general — becomes the first-order consideration. High-quality, hardware-aware forecasting is achievable under a 6GB VRAM budget, provided model selection is conditioned jointly on data structure and available compute.

5.2 Future works

Several directions follow from this research's findings and limitations:

- Confirmatory diagnostics: directly instrument attention-logit magnitudes during early STEEL training to empirically confirm the bounded-activation mechanism, evaluate robust, outlier-resistant scaling [59] as a potential fix for Informer/Autoformer, and complete the outstanding recurrent-baseline comparison on WTH and ETTh1.
- Efficiency for edge deployment: evaluate structured pruning, quantization [46, 56], and knowledge distillation [45], alongside the broader TinyML literature on embedded inference [61], to extend this thesis's hardware-aware framing toward genuine edge deployment beyond a single consumer GPU.
- Interpretability: apply established attention-interpretability methods [58, 64] to determine whether learned attention patterns correspond to physically meaningful periodicity, and whether early attention instability could be detected before a full STEEL-style collapse occurs.
- Domain generalisation: extend the accuracy–efficiency and data-suitability framework to traffic forecasting [52], financial forecasting [53], and time-series anomaly detection [54], each sharing structural properties examined in this thesis.
- Hybrid and privacy-aware architectures: evaluate hybrids combining bounded recurrent gating with self-attention, such as the Temporal Fusion Transformer [30], and federated learning approaches [55] for genuinely individual (rather than aggregated) household-level smart-meter data.
- Uncertainty and adaptation: extend this thesis's point forecasts to probabilistic forecasting [63], and evaluate continual or online learning [62] to handle distributional drift in deployed load patterns over time.

Collectively, these directions extend this thesis's central contribution — a hardware- and data-aware model-selection framework — toward the broader, ongoing challenge of reliable load forecasting under real-world deployment constraints [65], as power systems continue to integrate distributed and renewable generation.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in Proc. NeurIPS, 2017.

[2] International Energy Agency (IEA), World Energy Outlook 2023, IEA, Paris, 2023.

[3] A. L’Heureux, K. Grolinger, and M. A. M. Capretz, “Transformer-based model for electrical load forecasting,” Energies, vol. 15, no. 14, art. 4993, 2022.

[4] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne: OTexts, 2021.

[5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu, and Y. Zhang, “Short-term residential load forecasting based on LSTM recurrent neural network,” IEEE Trans. Smart Grid, vol. 10, no. 1, pp. 841–851, 2019.

[7] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient Transformer for long sequence time-series forecasting,” in Proc. AAAI, 2021, pp. 11106–11115.

[8] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition Transformers with auto-correlation for long-term series forecasting,” in Proc. NeurIPS, 2021, pp. 22419–22430.

[9] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with Transformers,” in Proc. ICLR, 2023.

[10] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are Transformers effective for time series forecasting?” in Proc. AAAI, 2023, pp. 11121–11128.

[11] L. Han, H.-J. Ye, and D.-C. Zhan, “The capacity and robustness trade-off: Revisiting channel independence in multivariate time series forecasting,” IEEE Trans. Knowl. Data Eng., vol. 36, no. 11, pp. 7129–7142, 2024.

[12] V. E. Sathishkumar, C. Shin, and Y. Cho, “Efficient energy consumption prediction model for a data analytic-enabled industry building in a smart city,” Building Research & Information, vol. 49, no. 1, pp. 127–143, 2021.

[13] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in Proc. ICLR, 2022.

[14] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.

[15] H. S. Hippert, C. E. Pedreira, and R. C. Souza, “Neural networks for short-term load forecasting: A review and evaluation,” IEEE Trans. Power Syst., vol. 16, no. 1, pp. 44–55, 2001.

[16] J. W. Taylor, “Triple seasonal methods for short-term electricity demand forecasting,” Eur. J. Oper. Res., vol. 204, no. 1, pp. 139–152, 2010.

[17] H. Lütkepohl, New Introduction to Multiple Time Series Analysis. Berlin: Springer, 2005.

[18] R. J. Hyndman, A. B. Koehler, R. D. Snyder, and S. Grose, “A state space framework for automatic forecasting using exponential smoothing methods,” Int. J. Forecasting, vol. 18, no. 3, pp. 439–454, 2002.

[19] R. F. Engle and C. W. J. Granger, “Co-integration and error correction: Representation, estimation, and testing,” Econometrica, vol. 55, no. 2, pp. 251–276, 1987.

[20] L. Sehovac and K. Grolinger, “Deep learning for load forecasting: Sequence to sequence recurrent neural networks with attention,” IEEE Access, vol. 8, pp. 36411–36426, 2020.

[21] A. Graves and J. Schmidhuber, “Framewise phoneme classification with bidirectional LSTM and other neural network architectures,” Neural Networks, vol. 18, no. 5–6, pp. 602–610, 2005.

[22] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in Proc. EMNLP, 2014, pp. 1724–1734.

[23] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” arXiv:1803.01271, 2018.

[24] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, “Modeling long- and short-term temporal patterns with deep neural networks,” in Proc. ACM SIGIR, 2018, pp. 95–104.

[25] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional Transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[26] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, “Convolutional sequence to sequence learning,” in Proc. ICML, 2017, pp. 1243–1252.

[27] P. Shaw, J. Uszkoreit, and A. Vaswani, “Self-attention with relative position representations,” in Proc. NAACL-HLT, 2018, pp. 464–468.

[28] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, “FEDformer: Frequency enhanced decomposed Transformer for long-term series forecasting,” in Proc. ICML, 2022, pp. 27268–27286.

[29] Y. Zhang and J. Yan, “Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting,” in Proc. ICLR, 2023.

[30] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, “iTransformer: Inverted Transformers are effective for time series forecasting,” in Proc. ICLR, 2024.

[31] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, and R. Jin, “CARD: Channel aligned robust blend Transformer for time series forecasting,” in Proc. ICLR, 2024.

[32] R. Ilbert et al., “SAMformer: Unlocking the potential of Transformers in time series forecasting with sharpness-aware minimization and channel-wise attention,” in Proc. ICML, 2024.

[33] P. Chen, Y. Zhang, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, B. Yang, and C. Guo, “Pathformer: Multi-scale Transformers with adaptive pathways for time series forecasting,” in Proc. ICLR, 2024.

[34] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, “Transformers in time series: A survey,” in Proc. IJCAI, 2023, pp. 6778–6786.

[35] L. Su, X. Zuo, R. Li, X. Wang, H. Zhao, and B. Huang, “A systematic review for transformer-based long-term series forecasting,” Artificial Intelligence Review, vol. 58, art. 80, 2025.

[36] Wu, S., Tang, Y., Zhu, Y., & et al. Session-based recommendation with graph neural networks[C].

In AAAI, 2019: 346-353.
[37] Liu, Q., Zeng, Y., Mokhosi, R., & Zhang, H. STAMP: short-Term attention/memory priority model for session-based recommendation[C]. In SIGKDD, 2018: 1831-1839. ACM.
[38] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.
[39] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
[40] L. Prechelt, “Early stopping — but when?” in Neural Networks: Tricks of the Trade, Springer, 1998, pp. 55–69.
[41] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in Proc. ICML, 2013, pp. 1310–1318.
[42] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” Int. J. Forecasting, vol. 22, no. 4, pp. 679–688, 2006.
[43] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” Information Sciences, vol. 191, pp. 192–213, 2012.
[44] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in Proc. ICLR, 2018.
[45] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in Proc. NeurIPS Deep Learning Workshop, 2015.
[46] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in Proc. ICLR, 2016.
[47] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.
[48] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. ICML, 2015, pp. 448–456.
[49] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” arXiv:1606.08415, 2016.
[50] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” IEEE Trans. Neural Networks, vol. 5, no. 2, pp. 157–166, 1994.
[51] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.
[52] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,” in Proc. ICLR, 2018.
[53] Z. Zhang, S. Zohren, and S. Roberts, “DeepLOB: Deep convolutional neural networks for limit order books,” IEEE Trans. Signal Process., vol. 67, no. 11, pp. 3001–3012, 2019.
[54] J. Xu, H. Wu, J. Wang, and M. Long, “Anomaly Transformer: Time series anomaly detection with association discrepancy,” in Proc. ICLR, 2022.
[55] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. AISTATS, 2017, pp. 1273–1282.
[56] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE/CVF CVPR, 2018, pp. 2704–2713.
[57] S. Serrano and N. A. Smith, “Is attention interpretable?” in Proc. ACL, 2019, pp. 2931–2951.
[58] P. J. Huber, “Robust estimation of a location parameter,” Ann. Math. Statist., vol. 35, no. 1, pp. 73–101, 1964.

[59] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA: O'Reilly Media, 2019.

[60] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in Proc. NeurIPS, 2017, pp. 6467–6476.

[61] T. Gneiting and M. Katzfuss, "Probabilistic forecasting," Annu. Rev. Stat. Appl., vol. 1, pp. 125–151, 2014.

[62] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?' Explaining the predictions of any classifier," in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[63] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," Int. J. Forecasting, vol. 32, no. 3, pp. 914–938, 2016.

Acknowledgments

I would like to express my deepest appreciation to everyone who has provided me with the support, guidance, and encouragement necessary to successfully complete this thesis. First and foremost, I would like to express my heartfelt appreciation to my family for their unwavering love, support, and belief in me throughout my academic journey. Their sacrifices, understanding, and encouragement have been instrumental in keeping me determined and focused on my goals.

I am gratefully indebted to my thesis advisor, Prof Lu Xin, for his invaluable guidance, mentorship, and wisdom throughout this research journey. Then my sincere gratitude goes to my colleagues, friends, and fellow researchers for their support, collaboration, and stimulating discussions.

Finally, I really thank everyone who has directly or indirectly contributed to my academic success and personal growth. This accomplishment would not have been possible without the collective effort and support of all those mentioned above.

Author: Amisi Lumbu
Espoir

September 2026, Nanjing